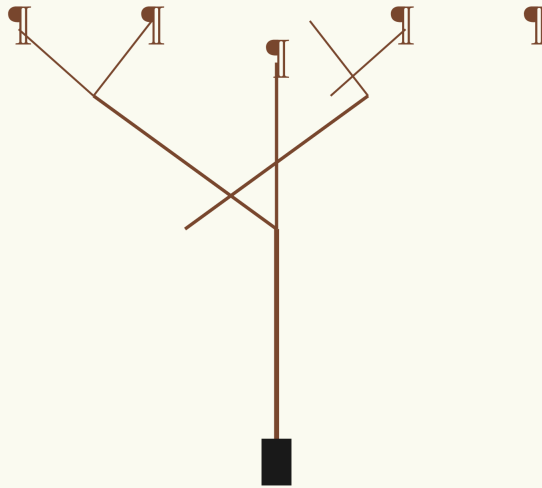




THE BOOK OF

INKHAVEN

An Author's Guide to the Literary TUI



every paragraph a leaf, every chord a branch

VLADIMIR ULOGOV

and the inkhaven contributors

VERSION 1.2.6 · 2026



The Book of Inkhaven — companion guide to the inkhaven TUI literary editor.

Source: <https://github.com/vulogov/blackInkhaven>

Typeset with Typst. Inkhaven binary versions are matched against book versions via the front-matter version string.

... to my wife. With love.

Contents

Prologue — Why this book exists	2
How to read it	2
Conventions	3
What this book is not	3
A note on AI	3
1 — What Inkhaven is	6
The layout	7
Local first — with one caveat	8
Typst as the typesetter	9
AI when you want it	9
What it is, in one sentence	10
2 — Installation and your first book	12
Initialise a project	13
Open the TUI	14
Add your first book	14
Open + write a paragraph	15
Build a PDF	16
Quit	16
3 — The project tree	18
Node kinds	19
Tree-pane chords	19
Anatomy of a paragraph row	20
Reordering	21
Multi-select	21
Folding for focus	21
4 — System books	24
The full set	25
Why they exist	25
Treating them as ordinary books	26
The Prompts book	26
5 — Configuring the project	28
What lives in the config	29
A real example — minimal	30
Editing safely	31
Project-level vs per-book	32

6 — Writing in Typst	34
Block-level markers	35
Inline markers	35
The skeleton inkhaven creates	36
Functions you'll actually use	36
When things go wrong	37
Markdown? Sort of.	37
7 — Editor workflow	40
Motion	41
Selection	41
Cut / copy / paste	42
Undo / redo	42
Search and replace	42
Split-edit (F4)	43
Save	44
8 — Saving and snapshots	46
F5 — create snapshot (with annotation)	47
F6 — the snapshot picker	48
Pre-restore safety	48
V — side-by-side diff	49
When to annotate	50
Snapshot lifecycle	50
9 — Status and writing goals	52
The status ladder	53
Cycling status	53
Status-filter modal	54
Writing goals — the goals stanza	54
Per-paragraph targets	55
Status hooks (Bund)	56
10 — Search and discovery	58
The search bar	59
Two engines	60
Fuzzy paragraph picker (`Ctrl+V P`)	60
Bookmarks (`Ctrl+V B / M`)	60
Help RAG (`F1`)	61
Search hooks (Bund)	61
11 — Backups and recovery	64
`inkhaven backup`	65

Auto-backup on exit	65
`inkhaven restore`	66
Recovery from drift	66
`inkhaven doctor`	67
12 — Exporting your book	70
From the TUI	71
From the CLI	71
Filter flags	72
Configuration — `output.extra_formats`	72
Per-paragraph extraction	72
EPUB metadata	73
Where everything lands	73
Render preview vs build	73
13 — Places and characters	76
Building entries	77
The highlight overlay	77
Stemming	78
RAG against an entry — `Ctrl+B P / C / A`	78
Lookup chords (no RAG)	79
Image references in entries	79
14 — Tags	82
Adding tags	83
Renaming tags project-wide	84
Tags in the tree pane	84
Tag-filtered export	84
Multi-select	84
Bund — `ink.tag.*` stdlib	85
Scrivener keyword import	85
15 — Wiki-links and backlinks	88
The four chords	89
Adding a link	89
Guards	90
Listing + removing	90
AI integration	91
Story-view edges	91
Bund — `ink.event.link_paragraph`	92
16 — The story view	94
The two chords	95

Book view	95
Paragraph mini view	96
Save the PNG	97
From Bund — <code>`ink.story.render`</code>	97
When the graph is too dense	98
17 — Story timeline	100
Enabling the timeline	101
Parser shapes	102
Where events live	103
Tree + editor visual cues	103
Adding events — three paths	104
Editing event timing (Ctrl+V Shift+I)	105
Linking events to manuscript paragraphs	106
CLI	107
Ctrl+V e — vertical event picker	107
Ctrl+V Shift+T — swim-lane view	108
Scope navigation	109
AI health critique	110
Bund — <code>`ink.event.*`</code>	111
Bund hooks	112
Orphans	112
18 — Configuring AI providers	114
The full set	115
Minimal configuration	115
Live switching — Ctrl+B L	116
Inference mode — Ctrl+B M / F10	116
AI scope — F9	117
Cost discipline	117
Privacy	117
Configuring per-provider	119
19 — The AI pane	120
Two layouts	121
Sending a prompt	122
Scope (F9) recap	122
Applying answers	122
Chat history	123
Search the chat (<code>`Ctrl+F`</code> in AI pane)	123
Selection mode (<code>`Ctrl+C`</code> in AI pane)	123

Status messages	124
Quick prompts (Ctrl+B G — Notes RAG)	124
20 — Prompts and the F7 grammar check	126
The prompt resolution chain	127
The Prompts book seeds	127
F7 — the grammar check	128
`g` to apply (grammar-aware)	128
Visual diff after apply	129
Customising the grammar prompt	129
Other prompt overrides	130
21 — Critique, memory, and the diff modal	132
F12 — critique (mode-aware)	133
Per-paragraph AI memory	133
The diff modal — accept / reject	134
Smart extraction — what lands in the buffer	135
Disabling the diff modal	136
Configuration recap	136
22 — AI for diagnostics and the timeline	138
Ctrl+F12 — explain the diagnostic at cursor	139
Customising the explain prompt	140
Timeline health critique (y / Y / Ctrl+Y)	140
What the audit asks for	140
Hook on diagnostic (Bund)	141
`ink.editor.set_cursor` for auto-jump	141
23 — Typst inside Inkhaven	144
Two engines	145
Why the in-process engine matters	145
Per-book Typst skeleton	146
`@preview` packages	146
Diagnostics from Typst	147
Doctor — check the setup	147
Font bundling	148
24 — Diagnostics and render preview	150
Gutter markers	151
F8 — diagnostics list	151
Ctrl+V N / Shift+N — next / previous diagnostic	152
Ctrl+F12 — AI explain	152
Ctrl+V R — render preview	153

Save flows	154
When NOT to use render preview	154
Configuration	154
`hook.on_diagnostic` recap	155
25 — Multi-format export	156
The five formats	157
From the CLI	157
Ctrl+B O — extra formats from inside the TUI	158
Per-paragraph quick extracts (`Ctrl+V 1/2`)	158
EPUB notes	159
LaTeX notes	159
Markdown round-trips	159
Filter combinations	160
26 — Importing existing work	162
Markdown / Typst directory tree	163
The F3 file picker	164
Scrivener import	164
Scrivener keywords → tags (1.2.6+)	165
Scrivener — what's NOT imported	165
Typst help reference	166
Importing the Book of Inkhaven into Help	166
27 — Theming and the cheat sheet	168
The theme stanza	169
Quick palette swap	169
Per-syntax colours	170
Lexicon overlay colours	170
Editor knobs (other)	171
The cheat sheet	171
Credits	172
Tutorial-by-tutorial colour examples	172
28 — Reassigning keys	174
The keys stanza	175
Chord syntax	175
Listing actions	176
Layered chord tables	176
Dynamic rebinding via Bund	176
Re-binding the prefix keys	177
F11 → Ctrl+F12 — the macOS workaround	177

29 — Bund scripting	180
Where Bund lives	181
Stack model	181
The `ink.*` stdlib	182
Hooks	183
Sandbox policy	184
A first hook	185
Where to learn more	185
Appendix A — Keybinding reference	188
Global (anywhere)	189
Tree pane	189
Editor pane	190
AI pane	191
Meta prefix (Ctrl+B)	191
Bund prefix (Ctrl+Z)	192
View prefix (Ctrl+V)	192
Inside the timeline view (Ctrl+V Shift+T)	193
Inside Ctrl+V R (render preview)	193
Appendix B — Configuration reference	194
Top-level	195
editor	195
llm — AI providers	195
ai (1.2.6+)	196
typst_compile	196
typst_page / typst_fonts / typst_layout	197
images	197
output	197
goals	198
timeline (1.2.6+)	198
backup	198
sound	199
scripting	199
keys	199
theme	200
Appendix C — Bund stdlib + hooks	202
Tree (ink.node.*, ink.tree.*)	203
Paragraph (ink.paragraph.*)	203
Tags (ink.tag.*, 1.2.6+)	204

Events (ink.event.*, 1.2.6+)	204
Story (ink.story.*, 1.2.6+)	204
Editor (ink.editor.*)	205
Search (ink.search.*)	206
Snapshots (ink.snapshot.*)	206
Filesystem (ink.fs.*)	206
AI (ink.ai.*)	206
Keymap (ink.key.*)	207
Pane + input	207
DB (ink.db.*)	207
Hooks (fire signatures)	207
Policy categories	208
About the Author	210
Why Inkhaven exists	211
A work of love	212
A note on cooperation	212
Where to find more	213

Why this book exists

Many writers have a process that survives them — a way of moving through a manuscript that the next book inherits without anyone teaching it. Inkhaven exists for that process: a quiet, keyboard-driven editor that holds the manuscript as a structured tree, talks to large language models without leaking your private prose, renders to Typst PDFs, and stays out of your way while you write.

This book is a guided tour through every feature, in the order a working novelist would actually meet them. It begins with installing the binary and writing a single paragraph; it ends with scripting the project from a custom Bund hook that checks your timeline for consistency on every save.

How to read it

Read in order if you are new. The early chapters are deliberately bare — open a project, type a sentence, save — because everything later assumes you’ve felt

those motions in your fingers. By the time you reach Part IV on world building you should be able to navigate without thinking about the chord chart.

Skip around if you've been using inkhaven for a while. Each chapter is self-contained: it tells you what's new in its corner of the system, lists the relevant chords, shows a figure of the screen the chord produces, and ends with a recap you can scan in a minute. Cross-references point you sideways when something interlocks.

Conventions

Keystrokes are written `Ctrl+S`, `F1`, `Ctrl+B` then `P` (the meta prefix needs two keypresses). Code blocks contain prose you type into a file or commands you run at a shell — the surrounding text says which.

Every figure is a placeholder: a numbered slot the book fills with a real terminal screenshot. The catalog in `Book/SCREENSHOTS.md` lists every figure, the chord state it captures, and the terminal size + theme it expects.

What this book is not

Not a Typst reference. The Typst language has its own excellent documentation at `typst.app/docs` we'll show you what the typesetter does when inkhaven sends prose to it, not how the typesetter itself works.

Not a tutorial on writing. We assume you bring the prose; inkhaven is the room you write it in.

Not exhaustive about Bund. Chapter 29 introduces the language and shows the inkhaven-specific `stdlib`; the canonical reference at `Documentation/Bund/BUND_TUTORIAL.md` goes deeper.

A note on AI

Inkhaven is built to use large language models as writing partners — for grammar review, critique, RAG search, and timeline auditing. The book describes those features honestly: when they help, when they get in the way, and how to turn them off when the prose needs your full attention.

Every AI surface is opt-in or scope-limited. Inkhaven never sends a paragraph to a remote provider without you pressing a chord. The full-screen AI mode (`Ctrl+B K`) and the per-paragraph memory (`ai.per_paragraph_memory`) are also opt-in.

But — and this is important — inkhaven does not provide inherent privacy when you use one of the cloud LLM providers (Gemini, Claude, OpenAI, DeepSeek, Grok). Every chord that talks to those providers sends prose to their servers under their terms of service. They may log it, train on it, or otherwise retain it. Chapter 18 covers this in detail.

For privacy, install a local Ollama instance and set `llm.default_provider: "ollama"`. Every non-LLM AI surface in inkhaven is already on-device; Ollama closes the last loop. If you write entirely without AI, the configured provider is irrelevant — none of inkhaven's non-AI features need an LLM.

Now — let's begin.

PART I — FOUNDATIONS

1

What Inkhaven is

Inkhaven is a single binary that, when you run it in a project directory,

paints a terminal user interface laid out as three panes in the middle, with a search input at the top and an AI prompt input at the bottom. Behind the panes sits a small database that stores every paragraph as a node in a tree, every snapshot, every wiki-link, every tag — and an indexer that makes the manuscript searchable both semantically (by meaning) and by exact text.

That's the whole shape. Everything else this book teaches you is a way of moving through that tree, sending parts of it to a language model, or rendering it to Typst.

The layout

The screenshot displays the Inkhaven terminal interface. At the top, there is a search input field. Below it, the interface is divided into three main panes: a Tree view on the left, an Editor in the center, and an AI prompt input on the right. The Tree view shows a hierarchical structure of nodes. The Editor pane displays a paragraph of text. The AI prompt input is at the bottom, with a status line beneath it. The interface is dark-themed and uses a monospaced font.

The layout: search input along the top; three panes in the middle — Tree (left), Editor (centre), AI (right); AI prompt input along the bottom; status line beneath. Modal pickers temporarily replace one or more of the centre panes.

The Search input (top) is dual-mode: type a query, hit Enter, and the result list overlays the tree pane.

The Tree pane (centre-left) shows the manuscript as a folder hierarchy. Books contain chapters; chapters contain subchapters or paragraphs; paragraphs contain prose.

The Editor pane (centre) holds the open paragraph. This is where you type. Saves go to disk and the database together; nothing waits in unsaved buffers when you Ctrl+S.

The AI pane (centre-right) holds the conversation with the language model. By default it's a chat history; in full-screen mode (Ctrl+B K) it grows to fill the screen when you want to think aloud with the model for a stretch.

The AI prompt input (bottom) is where you type prompts to send to the language model. Ctrl+I focuses it from any pane.

Modal pickers (Ctrl+V P fuzzy finder, Ctrl+B] tag picker, F6 snapshot picker, the swim-lane timeline view, the AI diff modal, ...) temporarily replace one or more panes; Esc returns to the previous layout.

Local first — with one caveat

Every byte you write lives in one folder. The database is DuckDB; the prose is Typst markup; the vectors live next to the database in a `vectors/` directory. You can rsync the whole project to a backup drive, push it to git, or carry it on a USB stick. The format is plain text + a few binary indices you can rebuild from the prose.

Inkhaven does not phone home, does not require an account, does not require an internet connection unless you ask the AI pane for help. The first time you run it, no analytics, no telemetry, no welcome video. Just a TUI.

PRIVACY WHEN USING AI

Inkhaven does not provide inherent privacy when you use one of the external LLM providers (Gemini, Claude, OpenAI, DeepSeek, Grok). Every prompt + RAG-attached paragraph travels to that provider's servers under their terms of service. They may log, train on, or otherwise retain what you send.

For increased privacy, set `llm.default_provider: "ollama"` and run a local Ollama instance. Every other inkhaven subsystem (RAG embedding, semantic search, snapshot diff, vector store) is already on-device — Ollama closes the last loop and keeps every byte on your machine. See Chapter 18 for setup.

Typst as the typesetter

Inkhaven manuscripts are written in Typst — a modern typesetting language designed for academic books and literary work. You don't need to learn Typst to use inkhaven; the editor accepts plain prose and the bundled templates handle layout. But the moment you want a fancy heading or a margin note, Typst is there.

IF YOU'VE USED LATEX

Typst is what LaTeX would look like if it had been designed in 2019 instead of 1985. Same idea (markup → professional typesetting), much friendlier syntax, faster compile. Inkhaven embeds the Typst compiler directly so you don't need to install it separately.

AI when you want it

The AI surface is the part that polarises. Inkhaven's position: AI is a writing partner, not a writing tool. You ask it for grammar review, for critique, for “what's weak here”, for help untangling a Typst diagnostic. It never edits your paragraph without an explicit accept step (`r` or `g` in the AI pane → diff modal → `a`).

You configure which LLM provider to talk to in `inkhaven.hjson`. Six providers ship in the binary: Gemini, Claude, OpenAI, DeepSeek, Grok, and Ollama (local). Switch live with `Ctrl+B L`.

If you don't want AI in your writing room at all, simply don't configure a provider. Inkhaven works fully without one.

What it is, in one sentence

Inkhaven is a TUI for writing a book. The tree is the book. The paragraphs are the prose. Everything else is a convenience you can ignore until you need it.

RECAP

- Single binary, single folder per project — local first.
- Four-pane TUI: Tree · Editor · AI · Search bar.
- Typst under the hood for typesetting; bundled compiler.
- AI is opt-in, scope-limited, and never auto-applies edits.
- Everything else is a layer you can engage or ignore.

PART I — FOUNDATIONS

2

Installation and your first book

There are three ways to install the binary, in increasing order of “I know what I’m doing”:

```
# 1. Pre-built binary via cargo-binstall (fastest)
cargo binstall inkhaven

# 2. Build from source (gets the latest commit)
cargo install --git https://github.com/vulogov/blackInkhaven

# 3. Direct download from the GitHub releases page
#   https://github.com/vulogov/blackInkhaven/releases
```

All three give you a single executable named `inkhaven`. Drop it on your `PATH` (e.g. `~/ .cargo/bin/` is on `PATH` after `cargo install`).

LINUX BUILD DEPENDENCY

Source builds on Linux need `libasound2-dev` at compile time (for the typewriter audio feedback). The prebuilt binaries don’t need it at runtime.

Initialise a project

A project is a directory. You initialise one with:

```
inkhaven init ~/Documents/MyBook
```

This creates the directory if it doesn’t exist, writes a default `inkhaven.hjson` (the project’s config), seeds the DuckDB metadata + vector indices, and pre-creates the seven system books (Notes, Research, Prompts, Places, Characters, Help, Typst). You’ll meet the system books in Chapter 4.


```

~/target/release/inkhaven init [facebook --force]
2024-06-23T21:13:08.478380Z INFO inkhaven::cli::init write project config path: /home/alex/inkhaven/inkhaven.json
2024-06-23T21:13:08.478380Z INFO inkhaven::cli::init write prompt library path: /home/alex/inkhaven/inkhaven_prompt.json
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_tree_add (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_tree_delete (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_tree_remove (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_tree_add_child (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_tree_remove_child (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_tree_move (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_tree_move_up (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_tree_move_down (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_tree_move_target (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_paragraph_add_child (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_paragraph_remove (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_db_sync (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_db_checkpoint (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_db_reload (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_kv_find (category keyval)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_kv_remove_key (category keyval)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_kv_update (category keyval)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_kv_delete (category keyval)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_editor_get (category editor_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_editor_insert (category editor_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_editor_delete (category editor_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_editor_delete_to_end (category editor_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_ai_list_history (category ai_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_ai_send (category ai_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_ai_set_system_prompt (category ai_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_editor_replace (category editor_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_editor_replace_all (category editor_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_ai_send_blocking (category ai_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_theme_set (category theme_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_typed_assemble (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_typed_serialize (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_typed_deserialize (category store_write)
2024-06-23T21:13:08.478380Z WARN inkhaven::scripting::posix deny ink_kv_write (category kv_write)
Shutting down inkhaven project at /home/alex/inkhaven
config: /home/alex/inkhaven/inkhaven.json
prompt: /home/alex/inkhaven/inkhaven_prompt.json
store db: /home/alex/inkhaven/inkhaven_store.db
metadata: /home/alex/inkhaven/inkhaven_metadata.db
books: /home/alex/inkhaven/inkhaven_books

```

Running `inkhaven init` shows the layout it just created — config path, books directory, metadata DB, vector store.

Open the TUI

```
cd ~/Documents/MyBook
inkhaven
```

The TUI paints — search input at the top, three panes in the middle (Tree · Editor · AI), AI prompt input at the bottom. Tree is empty (no user books yet, just the system ones).

Add your first book

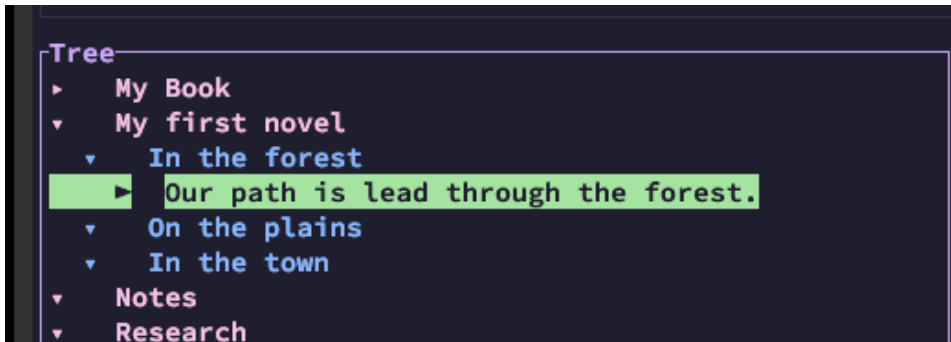
From the CLI, before going into the TUI:

```
inkhaven add book "MyFirstBook"
```

This creates a top-level **Book** node in the tree with slug **my-first-book**. Subsequent **add** calls hang chapters and paragraphs underneath:

```
inkhaven add chapter "Chapter 1" --parent my-first-book
inkhaven add paragraph "Opening scene" \
  --parent my-first-book/chapter-1
```

You can also do all of this from inside the TUI; the CLI version is just easier to script.



Tree pane after the three add commands: book → chapter → paragraph.

Open + write a paragraph

Re-launch `inkhaven`. In the Tree pane navigate (arrow keys) to “Opening scene” and press `Enter`. The Editor pane populates with the paragraph’s typst skeleton:

```
= Opening scene
```

Type a sentence. Press `Ctrl+S`. Status line reports “wrote N bytes”. That’s the loop — open a paragraph, write, save, move to the next.

ONE PARAGRAPH AT A TIME

Inkhaven holds exactly one paragraph open in the editor at any moment. Switching paragraphs autosaves the previous one. This is intentional: the unit of writing is a paragraph, and the database stores them individually so you can attach status, tags, snapshots, and AI memory at paragraph granularity.

Build a PDF

`Ctrl+B B` (“build the book”) tells Inkhaven to assemble every paragraph in the current user book into one Typst document and compile it to PDF. The PDF lands next to your project in `inkhaven-artefacts/<book-slug>/<book-slug>.pdf`.

```
Inkhaven · typst compile · done
✓ Build complete.
Book:    My first novel
Engine:  external · /opt/homebrew/bin/typst
Elapsed: 0s
Press any key to continue...
```

Building the book — splash + spinner while typst compiles. Cancellable with Esc.

Quit

`Ctrl+Q`. If the open paragraph is dirty, it autosaves before exit. Inkhaven also writes a backup automatically — see Chapter 11.

RECAP

- Three install paths; `cargo binstall inkhaven` is the fastest.
- Project = directory + `inkhaven.hjson` + DB + books.
- `inkhaven init <path>` creates everything; `inkhaven add` populates the tree.
- One paragraph open at a time; switching autosaves.
- `Ctrl+B B` builds the book; PDF lands in `inkhaven-artefacts/`.
- `Ctrl+Q` quits; dirty paragraph autosaves.

PART I — FOUNDATIONS

3

The project tree

The tree pane is the manuscript's spine. Every piece of prose, every script, every note hangs from it. Understanding the kinds of nodes, the parent-child rules, and how to reorder them is the first move toward feeling comfortable in inkhaven.

Node kinds

Chord	What it does
Book	Top-level container. A project can have multiple. System books (Notes, Help, ...) sit alongside user books.
Chapter	First-level container inside a book.
Subchapter	Optional second-level container inside a chapter.
Paragraph	A leaf. Carries the prose. Has metadata: status, tags, word target, links, snapshots.
Script	Bund (.bund) leaf. Stores executable code instead of prose. See Chapter 29.
Image	PNG / JPG leaf. Referenced by paragraphs through <code>`#image()`</code> typst calls.

The hierarchy rule: Book > Chapter > Subchapter > leaves. Books can hold paragraphs directly (no chapter), chapters can hold paragraphs directly (no subchapter), but you can't nest two chapters or put a book inside another book.

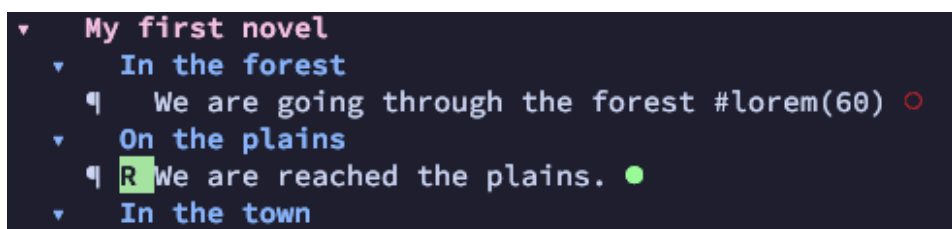
Tree-pane chords

With focus on the Tree pane (`Ctrl+2`):

Chord	What it does
↑ / ↓ / PgUp / PgDn	Navigate.
← / →	Collapse / expand a branch.
Enter	Open the paragraph (or focus an image / script).

Chord	What it does
F2	Rename the cursor node.
B / C / A / +	Add book / chapter / subchapter / paragraph as a child of the cursor.
V / S / P	Insert chapter / subchapter / paragraph as a sibling AFTER the cursor.
D	Delete the cursor's branch (confirm prompt).
-	Delete the cursor paragraph (no children — quick path).
U / J	Reorder up / down among siblings.
Z / X	Collapse cursor's subchapter / collapse every branch.
Space	Multi-select toggle (see Chapter 14 for the picker workflows that use marks).

Anatomy of a paragraph row



A paragraph row carries five things at a glance: indent (depth), kind glyph (¶), status letter (N/F/R/...), title (truncated), and tag pips (#draft, #weather).

The pieces from left to right:

Chord	What it does
Indent	Two spaces per depth level.
Glyph	¶ for paragraph, λ for script, [?] for image, ▼/► for expanded/collapsed containers, ► for the open paragraph.

Chord	What it does
Status letter	N (Napkin), F (First), ... R (Ready). Spaced reserved when status = None. See Chapter 9.
Title	Truncated past 36 chars.
Progress pip	Tiny circle (○ ◐ ◑ ◒ ●) when the paragraph has a word target. See Chapter 9.
Tag pips	Up to two #tag chips + a +N count. See Chapter 14.

Reordering

U (up) and **J** (down) move the cursor node among its siblings. The shift is one position per press; long moves take repetition. The change is immediate — there’s no “save the tree” step.

When you rename a node with **F2**, both its title AND its slug update. The slug is what shows up in URLs and file paths; it’s auto-generated from the title (kebab-case) but you can edit it independently in the metadata.

Multi-select

Space toggles whether the cursor node is “marked”. Marked nodes get a  glyph; status badges and reorder actions apply to all of them at once. This is covered in detail in Chapter 14 alongside the tag workflows that take advantage of it.

Folding for focus

A 200-paragraph manuscript is overwhelming in a flat tree. **X** collapses every expanded branch; **Z** collapses just the cursor’s enclosing subchapter; **←** collapses the cursor’s direct branch. The opposite (expand) is **→**.

TIP

Combine `X` with the fuzzy paragraph picker (`Ctrl+V P`, Chapter 10): collapse everything, then jump to whatever you actually want to work on. Reduces visual noise to about one row per chapter title.

RECAP

- Tree kinds: Book → Chapter → Subchapter → Paragraph / Script / Image.
- Tree-pane chords mirror file-manager intuitions: arrows, Enter, F2, D.
- `U / J` reorder siblings; `B / C / A / +` create children; `V / S / P` create siblings.
- Multi-select with `Space` powers project-wide picker workflows.
- `X` collapses everything — combine with the fuzzy picker for focused work.

4

System books

Every inkhaven project ships with eight pre-created books that live alongside your manuscript. They handle a specific role in the writing pipeline; the tree pane shows them after your user books.

The full set

Chord	What it does
Notes	Free-form notes, outlines, side thoughts. The default search target for `Ctrl+B G` (note-RAG).
Research	World-research / source material. Searchable by F1 if you add it to the Help index.
Prompts	AI prompt templates. Override the embedded defaults by renaming a paragraph to drop its `.example` suffix.
Places	Locations referenced by the manuscript. Powers the yellow place-highlight overlay in the editor.
Characters	Character cards. Powers the cyan character-highlight overlay.
Artefacts	Physical objects, magic items, recurring symbols. Same overlay machinery as Places / Characters.
Typst	Read-only. Bundled typst reference that powers F1 typst-help queries.
Help	Read-only. Bundled how-to text. F1 RAG searches here.

There's also a per-user-book Timeline chapter that materialises lazily on first `event add` (Chapter 17). It's tagged `system_tag: book_timeline` so renames don't break it.

Why they exist

Each system book pays its rent in features:

- Prompts: When `Ctrl+B G` (grammar check), `Ctrl+F12` (explain diagnostic), or `F12` (critique) sends a prompt to the AI, it resolves the prompt template by looking up a paragraph in this book first, then falling back to `prompts.hjson`, then to the embedded default.
- Places / Characters / Artefacts: Inkhaven scans the open paragraph against the titles in these books and highlights matches inline. `Ctrl+B P` / `Ctrl+B C` / `Ctrl+B A` open a RAG flow against the matched entry. See Chapter 13.
- Notes: `Ctrl+B G` (note RAG) sends a query against every note’s content with the AI hooked up — useful when you’ve scribbled something three weeks ago and can’t remember where.
- Typst / Help: `F1` query against either one.

Treating them as ordinary books

System books look special in the tree (a `(system)` suffix

1. they’re protected from `D` delete on the book itself) but

their paragraphs are ordinary paragraphs. Tag them, link them, give them word targets — the same metadata stack applies. So a Character card can have status `Final`, a target of 800 words, and tags `[protagonist, main, draft]`.

IF YOU IMPORT A SCRIVENER PROJECT

Inkhaven’s importer maps Scrivener’s “Characters” / “Places” / “Notes” folders to the corresponding system books automatically. See Chapter 26.

The Prompts book

The Prompts book gets its own chapter (Chapter 20) but the short version: on `inkhaven init`, five paragraph templates get seeded automatically with the `.example` suffix:

Prompts/

- |— grammar-check.example
- |— explain-diagnostic.example
- |— critique-edit.example
- |— critique-changes.example
- |— timeline-health.example

Open one (`Enter`), edit the body, then rename (`F2`) to drop the `.example` suffix. From that moment inkhaven uses your version instead of the embedded fallback.

RECAP

- Eight system books: Notes, Research, Prompts, Places, Characters, Artefacts, Typst, Help.
- Each pays for itself by powering a specific feature (RAG, lexicon highlight, prompt resolution).
- System books are protected at the book level but their paragraphs are ordinary.
- The lazy Timeline chapter is per-book — Chapter 17.
- Customise AI prompts by editing the `.example` seeds in the Prompts book and dropping the suffix.

5

Configuring the project

Every project has one file that sets the rules for the rest:

`inkhaven.hjson`. HJSON is JSON with comments and trailing-comma forgiveness — friendly to write by hand.

The file lives at the project root. Edit it with any text editor; changes take effect on the next inkhaven launch.

What lives in the config

The default file is heavily commented. Major blocks:

Chord	What it does
language	Primary writing language. Drives stemmer choice + the F7 grammar prompt default.
editor	Word-wrap, autosave idle, status-bar shape, sounds, theme. See Chapter 27.
llm	AI provider configuration. Six providers; one default. See Chapter 18.
ai	AI behaviour: per-paragraph memory, diff review on apply, prompt-example seeding. See Chapter 21.
typst_compile	External CLI vs in-process engine; fonts;

Chord	What it does
	package resolution. See Chapter 23.
<code>typst_page / typst_fonts / typst_layout</code>	Page size, fonts, par layout for the bundled templates.
<code>output</code>	Multi-format export defaults (epub meta-data, markdown variant, ...). See Chapter 25.
<code>goals</code>	Word-count targets + streak tracking. See Chapter 9.
<code>timeline</code>	Story-timeline feature: enable, default track, calendar config. See Chapter 17.
<code>backup</code>	Auto-backup-on-exit destination + max-age. See Chapter 11.
<code>images</code>	Terminal-graphics support (kitty / iterm2 / sixel / half-block fallback).
<code>scripting</code>	Bund sandbox policy + bootstrap script. See Chapter 29.
<code>keys</code>	Chord rebinding overlay. See Chapter 28.
<code>theme</code>	Colour palette. See Chapter 27.

A real example — minimal

```

{
  language: "english"

  llm: {
    default_provider: "ollama"
    ollama: { model: "qwen2.5:7b" }
  }

  editor: {
    typewriter_sounds: false
  }

  typst_compile: {
    engine: "inprocess"
  }

  timeline: {
    enabled: true
    calendar: { preset: "gregorian" }
  }
}

```

This minimal stanza: English as the language, local Ollama for AI (no internet required), no typewriter audio, the bundled in-process typst compiler, and the timeline feature enabled with gregorian dates. Three minutes of config and you have a working setup.

Editing safely

The TUI doesn't reload the config on the fly. Edit `inkhaven.hjson`, quit (Ctrl+Q), launch again.

The full reference lives at `Documentation/CONFIGURATION.md`. Appendix B in this book is an abbreviated reference card — every knob and its default, no examples.

SAFE TO BREAK

If you make `inkhaven.hjson` invalid, `inkhaven` refuses to start with a clear error pointing at the line. Your prose is never at risk; the config is a separate file from the database. Worst case: revert your edit, launch again.

Project-level vs per-book

Most configuration is project-level — one file for the whole directory. A few things are per-book:

Chord	What it does
Book title / slug	Set with <code>`inkhaven add book`</code> or F2 on the book node.
Per-book typst skeleton	<code>`books/<slug>/globals.typ`</code> , <code>`settings.typ`</code> , <code>`index.typ`</code> — created automatically.
Per-book Timeline chapter	Lazily created on first <code>`event add`</code> for that book.

The per-book typst files (`globals.typ`, `settings.typ`, `index.typ`) are where you put book-specific `#set` and `#show` rules. The build pipeline imports them into the assembled document automatically.

RECAP

- `inkhaven.hjson` at the project root is the single config file.
- Comments + trailing commas allowed (HJSON, not strict JSON).
- Restart `inkhaven` to pick up changes.
- Appendix B summarises every knob; full reference in `Documentation/CONFIGURATION.md`.
- Per-book customisation lives in `books/<slug>/globals.typ` + `settings.typ`.

PART II — THE EDITOR

6

Writing in Typst

You don't have to learn Typst to write in inkhaven. Plain prose just

works — the bundled templates handle every typographic decision. But the moment you want bold, emphasis, or a section break, Typst is the syntax.

This chapter is a survival kit: the dozen Typst markers you'll actually meet writing prose. The full Typst language goes much further (math, plots, custom functions); see `typst.app/docs` when you need it.

Block-level markers

Chord	What it does
<code>= Title</code>	Top-level heading (chapter).
<code>== Title</code>	Second level (subchapter).
<code>=== Title</code>	Third level (section inside a chapter).
<code>- item</code>	Bulleted list item.
<code>+ item</code>	Numbered list item (the typst form; <code>`1. item`</code> also works).
<code>> quote</code>	Block quote — single line OR several with leading <code>`>`</code> on each.
<code>// comment</code>	Comment. Skipped by the typesetter; visible in the editor.

Paragraph break = blank line. Single newlines inside a paragraph are treated as soft wraps and joined.

Inline markers

Chord	What it does
<code>*bold*</code>	Bold text — single asterisks.
<code>_italic_</code>	Italic — single underscores.
<code>`code`</code>	Inline code — backticks.

Chord	What it does
"smart quotes"	Curly quotes when the editor's language is set; straight when set to `none`.
---	Em dash. `--` is en dash.
[link](url)	Hyperlink in markdown form (Typst accepts both `#link()` and the bracket form).
#emph[text]	Function form of italic — useful inside complex content.
#strong[text]	Function form of bold.

The skeleton inkhaven creates

When you create a paragraph (+ in the tree or `inkhaven add paragraph`), inkhaven writes a tiny skeleton:

```
= My paragraph title
```

The `= Title` line is what gets used as the slug seed + appears at the top of the rendered output. Add prose below the blank line. The title line is yours to edit; F2 rename in the tree updates it.

Functions you'll actually use

A handful of Typst function calls cover most prose needs:

```
#image("../images/cover.jpg", width: 80%)

#footnote[A quick aside.]

#emph[delicate emphasis] vs *strong emphasis*

#text(style: "italic", "rendered italic without the inline
_")
```

```
#pagebreak()
```

The `#` prefix is Typst’s “this is a function call” marker. Anything you write in pure prose without `#` is treated as prose; everything with `#` is computed.

IMPORTS HAPPEN AUTOMATICALLY

Inkhaven assembles the book by walking the tree, pulling each paragraph’s `.typ` body, and wrapping with the book’s `index.typ` + `settings.typ` + `globals.typ`. So functions you define in `globals.typ` are usable from any paragraph without an `#import` line.

When things go wrong

Type something Typst doesn’t like — `#undefined_function()` or an unmatched `*` — and the editor’s gutter shows a red ● on the offending line (1.2.6+). Chapter 24 covers the diagnostic surface in depth.

The render preview (`Ctrl+V R`, Chapter 24) lets you see what a paragraph will look like in the final book without running the full build.

Markdown? Sort of.

Inkhaven’s AI pane talks markdown by default — code blocks in answers use triple backticks, headings use `#`, lists use `-` or `*`. When you apply an AI answer to a paragraph via `r / i / t / b`, inkhaven converts markdown to Typst on the fly. It’s not lossless but covers the basics: `# H` becomes `= H`, `**bold**` becomes `*bold*`, fenced code stays a code block.

If you write your own prose in markdown habits, that’s fine — most of it round-trips through the Typst engine cleanly.

RECAP

- Plain prose works. Typst markup is opt-in for the typographic moments.
- = == === headings; **bold**; *_italic_*; `code` ; --- em dash.
- `#function[arg]` for typst calls; `#image()` , `#footnote[]` , `#emph[]` .
- = Title line at the top of each paragraph is what inkhaven uses; F2 renames it.
- Diagnostics (gutter ●) and render preview (Ctrl+V R) catch problems early.

PART II — THE EDITOR

7

Editor workflow

The editor pane is a `tui-textarea` widget with inkhaven's overlays painted on top. Movement is keyboard only — the chord muscle memory is the same as `nano` / `micro` / VS Code's basic shortcuts, with a few inkhaven specifics.

Motion

Chord	What it does
Arrow keys	Character / line.
Home / End	Beginning / end of line.
Ctrl+Home / Ctrl+End	Top / bottom of paragraph.
Ctrl+← / Ctrl+→	Previous / next word.
PgUp / PgDn	Scroll a page.
Ctrl+G	Jump to line — type a number, Enter.

Selection

Chord	What it does
Shift+arrow	Extend selection one character / line.
Shift+Ctrl+arrow	Extend by word / paragraph.
Ctrl+A	Select all.
Ctrl+Shift+A	Select current paragraph (within the buffer — paragraph here means a block of prose, not the inkhaven Paragraph node).
Esc	Drop selection.

Cut / copy / paste

Chord	What it does
Ctrl+X	Cut to system clipboard.
Ctrl+C	Copy to system clipboard.
Ctrl+V	Note: in inkhaven `Ctrl+V` is the view-prefix chord (see Chapter 16). For paste, use the terminal's own paste binding (Cmd+V on macOS, Shift+Insert on Linux).

ON CTRL+V

Inkhaven reclaims Ctrl+V as the “View” prefix because every modern terminal already handles paste via OS-level bindings. If you can’t paste, the terminal — not inkhaven — is intercepting your shortcut.

Undo / redo

Chord	What it does
Ctrl+Z	Undo. NOTE: in inkhaven `Ctrl+Z` is the Bund prefix (Chapter 29). Use the editor-pane's own undo via the inline chord (in `tui-textarea`).
Ctrl+R or Ctrl+Y	Redo (depending on terminal).

The undo stack lives in the editor widget, not the database. Closing the paragraph clears the undo history — once you switch away, undo no longer reaches back. Snapshots (Chapter 8) are the durable rollback path.

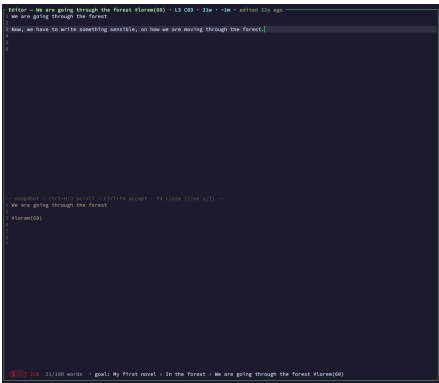
Search and replace

Chord	What it does
Ctrl+F	In-buffer find. Type, Enter, n / N walk hits.
Ctrl+H	In-buffer find-and-replace. Y/n/a per hit.
Ctrl+/ 	Focus the project-wide search input at the top of the screen.

Project-wide search (**Ctrl+/**) is a different beast — covered in Chapter 10 with semantic + full-text strategies.

Split-edit (F4)

F4 toggles split-edit mode. The editor pane splits in half: left side shows the most recent snapshot, right side holds your live buffer. Useful when you want to keep the previous version visible while you rewrite.



Split-edit (F4) — left half is the last snapshot, right half is the live buffer. Ctrl+H/J scroll the snapshot.

Chord	What it does
F4	Toggle split-edit.
Ctrl+H / Ctrl+J	Scroll the snapshot pane independently.
Ctrl+F4	Accept the snapshot's current scroll position as the new baseline (saves the right pane as a snapshot).

The F12 critique chord (Chapter 21) behaves differently when split-edit is active: the AI compares snapshot vs current and reports on the changes you made.

Save

`Ctrl+S` saves. Save writes:

1. The file on disk (`books/<book-slug>/.../<paragraph-slug>.typ`).
2. The metadata DB record (modified time, word count).
3. The vector store (re-embeds the prose for semantic search).
4. Fires `hook.on_save(uuid)` (Chapter 29 covers hooks).

Autosave: the editor's idle timer (configurable in `editor.autosave_idle_seconds`, default 30s) saves automatically. Switching paragraphs also saves.

RECAP

- Standard motion / selection / find / replace chords; `Ctrl+V` is the view prefix, not paste.
- Undo lives in the editor widget; snapshots are the durable rollback path.
- `F4` enables split-edit (snapshot vs live).
- `Ctrl+S` saves to disk + DB + vectors; autosave on idle + paragraph switch.
- Project-wide search (`Ctrl+/>`) is Chapter 10.

8

Saving and snapshots

Saves are atomic. Snapshots are deliberate. The distinction matters: save

commits your current paragraph to the on-disk file + database every time you press Ctrl+S; snapshot makes a versioned copy you can return to.

This chapter walks through both, plus the 1.2.6 annotation prompt that turns a wall of timestamp-only snapshots into a labelled revision history.

F5 — create snapshot (with annotation)

F5 (Editor scope) opens a small annotation prompt over the editor:

```
Snapshot annotation — F5  
  
Snapshot `We are going through the forest #lorem(60)` — annotation:  
> Our first steps|  
  
Enter commits (empty = no note) · Esc cancels
```

F5 — annotation prompt. Type a note describing this version, Enter commits, Esc cancels.

Chord	What it does
F5	Open the annotation prompt + capture the current buffer.
Type	Build up the annotation text.
Enter	Commit the snapshot.
Enter (empty)	Commit without an annotation — same as the pre-1.2.6 path.
Esc	Cancel — no snapshot written.

The buffer to snapshot is captured at prompt-open time, so you can keep typing in the editor while the prompt is open without changing what gets saved.

Annotations live in the snapshot's metadata, not the body. They show up in the F6 picker as italic cyan second lines under the row.

F6 — the snapshot picker

F6 lists every snapshot of the open paragraph:

```
Snapshots - We are going through the forest #Lorem(60)
2026-05-23 16:19:20 -0600 21w We are going through the forest
2026-05-23 16:18:17 -0600 21w We are going through the forest
    ♡ We are really going.
2026-05-23 16:16:54 -0600 21w We are going through the forest
    ♡ Our first steps
```

F6 picker. Annotated snapshots show their note on a second line; un-annotated ones are single-line.

Chord	What it does
↑ / ↓	Move cursor.
Home / End	Newest / oldest.
Enter	Pre-restore safety snapshot of the current buffer + restore the selected one.
V	Side-by-side diff: snapshot vs live buffer (see below).
D / Del	Confirm + delete the selected snapshot.
Esc	Close the picker.

Pre-restore safety

When you press Enter on a snapshot the system snapshots your current buffer FIRST (un-annotated), then loads the selected one. So you can always

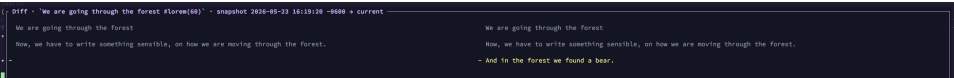
step back from a restore — the “undo” of a restore is another restore from the auto-saved pre-restore snapshot.

SAFETY NET

Inkhaven never overwrites your live work silently. If you’ve made changes since the last save, the auto-save before the restore captures them. If you restore the wrong snapshot, F6 → arrow keys to the new pre-restore entry → Enter recovers.

V — side-by-side diff

V inside the F6 picker opens a snapshot diff view: left side is the snapshot, right side is your current buffer. Use it to sanity-check what you’re about to restore.



Snapshot diff (‘V’ in F6). Left = snapshot; right = current buffer. Coloured markers show insertions / deletions per line.

Chord	What it does
↑ / ↓ / PgUp / PgDn	Scroll synchronised.
Esc	Back to the F6 picker.

When to annotate

Not every F5 fire deserves a note. The decision-point snapshots earn their labels:

- Before a structural rewrite — “before the lighthouse rewrite”
- After a complete draft — “first complete draft”
- A version that landed well — “version Maria loved”
- A milestone — “submission draft v1”
- A risky experiment — “trying second-person POV”

Skip annotations for the noisy mid-revision F5s. The un-annotated ones are still there; they just don’t contribute label noise.

Snapshot lifecycle

Snapshots are `bdslib` documents with `kind: "snapshot"` and a `parent_id` back-reference to the paragraph. They DO NOT appear in semantic search (we use `add_document_no_embed`) so a paragraph with 50 revisions doesn’t pollute search results. They DO travel with backups (Chapter 11).

Deleting a paragraph deletes its snapshots automatically via the cascade in `Store::delete_subtree`.

RECAP

- F5 snapshots with an optional one-line annotation. Empty annotation = un-annotated.
- F6 picker; Enter restores; V diffs; D deletes; pre-restore safety snapshot fires automatically.
- Save and snapshot are separate concepts: save = current state to disk; snapshot = versioned copy.
- Snapshots stay out of semantic search; they DO travel with backups.

9

Status and writing goals

Every paragraph carries two pieces of metadata that make a manuscript feel like a working project rather than a pile of text: its status on a workflow ladder and an optional word-count target. Both are visible at a glance in the tree pane.

The status ladder

Seven rungs, from least to most complete:

Chord	What it does
None	Default — unstated or unclaimed.
Napkin	A sketch or rough outline. The bones of an idea.
First	First draft — words on the page.
Second	Second pass — structure intact, prose polished.
Third	Third pass — pacing, voice, line-level fix.
Final	Final edit — ready for proof.
Ready	Ready to submit / publish.

The tree pane shows the status as a single letter (N, F, S, ...) coloured by rung. Books / chapters / subchapters roll up the deepest-rung paragraph beneath them.

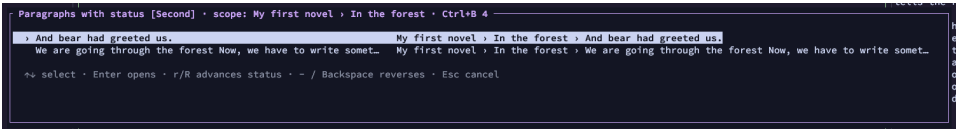
Cycling status

`Ctrl+B R` (Editor scope) cycles the open paragraph's status forward through the ladder. `Shift+Ctrl+B R` cycles backward. The tree pane updates immediately.

The same cycle works on the tree-cursor's paragraph from tree focus (`Ctrl+B R` in tree scope).

Status-filter modal

`Ctrl+B 1` through `Ctrl+B 7` open a project-wide filter modal keyed to one rung:



Ctrl+B 4 — every paragraph at Status:Second across the project. Enter opens; n/N walk in tree order.

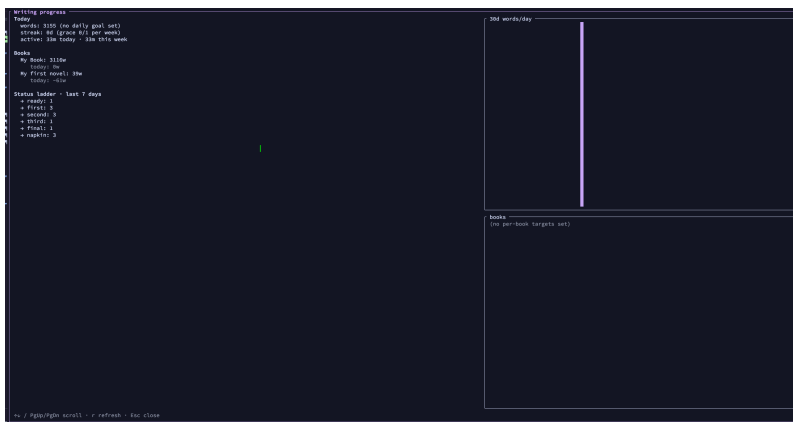
Chord	What it does
<code>Ctrl+B 1</code>	Filter to Napkin.
<code>Ctrl+B 2</code>	First.
<code>Ctrl+B 3</code>	Second.
<code>Ctrl+B 4</code>	Third.
<code>Ctrl+B 5</code>	Final.
<code>Ctrl+B 6</code>	Ready.
<code>Ctrl+B 7</code>	None (unstated).
<code>Enter</code>	Open the cursor paragraph in the editor.
<code>Esc</code>	Close.

Writing goals — the goals stanza

In `inkhaven.hjson`:

```
goals: {
  daily_words: 800          # baseline target
  morning_baseline: true    # measure from your first
                             # save of the day
  streak_grace: 1           # how many "rest days" the
                             # streak survives

  per_book_deadline: {
    "my-first-book": "2026-08-31"
  }
}
```



Ctrl+V G — progress modal. Today's words, current streak (with grace), per-book burn-down to deadline.

Ctrl+V G opens the progress modal. Per-day word counts, streak status, deadline burn-down, and a per-book rollout.

Per-paragraph targets

A target is a word count you want the paragraph to hit:

Chord	What it does
Ctrl+V G then T	Set the open paragraph's target (in the progress modal).

Chord	What it does
Ctrl+V T	Same shortcut from anywhere — opens a small number prompt.

The target shows up in the tree pane as a tiny progress pip (○ ● ● ● ●) coloured by completion (red at 0% → green at 100%+).

Status hooks (Bund)

Two Bund hooks fire on status events:

```
"hook.on_status_promoted" {
  // ( uuid new_status -- )
  drop drop
  // Side-effects when a paragraph is promoted up the
  ladder.
} register
```

```
"hook.on_goal_hit" {
  // ( word_count_today -- )
  drop
  // Maybe play a sound, push a notification, ...
} register
```

Useful for ergonomic touches — a typewriter ding when you hit the daily goal, an auto-tag when a paragraph promotes to Ready, an auto-snapshot when a paragraph promotes to Final.

RECAP

- Seven-rung status ladder: None → Napkin → First → Second → Third → Final → Ready.
- Ctrl+B R cycles forward; Shift+R cycles back.
- Ctrl+B 1..7 filters the project to one rung.
- goals: stanza controls daily target + streak + deadlines.

- Per-paragraph word targets render as a tree-pane progress pip.
- `hook.on_status_promoted` and `hook.on_goal_hit` enable rituals.

10

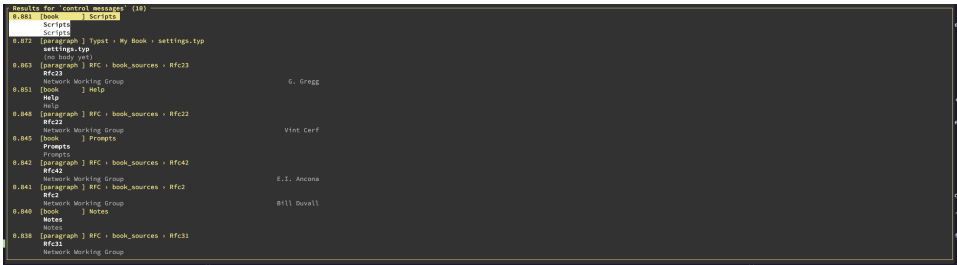
Search and discovery

Every paragraph you save gets embedded into a vector store. The result:

inkhaven’s search is dual — exact text matching AND semantic similarity. You don’t have to remember the words; you can search for the idea.

The search bar

Ctrl+/ focuses the search input at the top of the screen. Type, Enter. Results overlay the tree pane.



Search results overlay — semantic similarity scores on the left, exact-text match icon on the right. Arrows + Enter open.

Chord	What it does
↑ / ↓	Move cursor.
Enter	Open the cursor paragraph in the editor.
Esc	Close the overlay — tree pane returns.
Tab	Toggle between semantic and exact-match modes (default: semantic first).

Two engines

The semantic engine uses fastembed locally (no network calls) to embed your query, then queries the vector store via HNSW for the K nearest matches. The exact engine is plain substring search via the metadata DB. Both run in parallel; results merge by score.

The first time you launch inkhaven, the embedding model downloads (120 MB; sits in `~/ .cache/fastembed/`). After that, everything is offline.

Fuzzy paragraph picker (`Ctrl+V P`)

A second navigation tool — a popup picker over every paragraph title in the project:

Chord	What it does
Ctrl+V P	Open the picker.
Type	Fuzzy-match against title + slug-path.
Enter	Open the selected paragraph.
Esc	Close.

Faster than the search bar when you remember the title. Slash commands (`/recent` , `/bookmarks`) reorder the list.

Bookmarks (`Ctrl+V B / M`)

Chord	What it does
Ctrl+V B	Toggle bookmark on the open paragraph.
Ctrl+V M	Open the bookmark picker.

Bookmarks are stored on the node (`Node.bookmark: bool`). The picker shows every bookmarked paragraph in tree order; Enter opens, D removes the bookmark.

Help RAG (`F1`)

`F1` opens a query pane against the Help book — every paragraph in the system Help book gets retrieved by semantic similarity and stuffed into the AI context. Useful for “how do I do X in inkhaven” questions when you forget the chord.

After you import this Book of Inkhaven into the Help book (see Chapter 26), `F1` covers the full user surface.

F1 = ALWAYS-ON DOCS

Once the Help book is populated, `F1` is the fastest path from “I want to do something” to “found the chord”. Your Help RAG can index this book, the inkhaven tutorials, and the reference docs all at once.

Search hooks (Bund)

`ink.search.text` (Bund stdlib) lets scripts query the search engine directly:

```
"the storm came" 10 ink.search.text    // ( query limit --  
list )
```

Returns a list of hit hashes with `id`, `title`, `score`, `kind`, `document`. Useful for hooks that react to writing patterns (“oh, you mentioned ‘storm’ — relevant scenes: ...”).

RECAP

- `Ctrl+/` focuses the search bar; semantic + exact in parallel.
- `Ctrl+V P` is the fuzzy paragraph picker (faster when you remember the title).
- `Ctrl+V B / M` for bookmarks.
- `F1` opens Help RAG — RAG over the Help book.
- `ink.search.text Bund stdlib` for programmatic queries.
- First launch downloads the fastembed model; everything offline after.

11

Backups and recovery

Inkhaven's backup story is dead-simple zip archives of your project directory. There's nothing proprietary; you could untar one in any directory and have a working project. Backups travel.

`inkhaven backup`

```
inkhaven backup
# → ./inkhaven-backups/<project-basename>/<project>-
YYYYMMDD-HHMM.zip

inkhaven backup --out /backups/inkhaven/
# → /backups/inkhaven/<project>-YYYYMMDD-HHMM.zip
```

The zip contains everything: `books/`, `metadata.db`, `vectors/`, `inkhaven.hjson`, and any artefacts you've generated. Snapshot history is in `metadata.db` so it travels too.

Auto-backup on exit

When you `Ctrl+Q` and the last backup is older than `backup.max_age`, inkhaven runs a backup automatically before the TUI tears down. Configured in `inkhaven.hjson`:

```
backup: {
  out_dir: ""                # empty → sibling
                              # `inkhaven-backups/
  <project>/`
  max_age: "24h"             # how long since the last
                              # backup before exit
  triggers one
}
```

`max_age: "0s"` disables auto-backup. The manual command still works.

```
Inkhaven · backup · done
✓ Backup complete.
Project: /Users/gandalf/testbook
Wrote:   /Users/gandalf/inkhaven-backups/testbook/blackinkhaven_20262305_224615.zip
Press any key to continue...
```

Ctrl+Q with stale backup — splash + progress bar while the zip is written. Esc cancels.

`inkhaven restore`

```
inkhaven restore --from inkhaven-backups/my-book/my-book-20260520-1432.zip
inkhaven restore --from ... --target /tmp/my-book-test
```

Without `--target` the restore is to the current project directory (refusing to overwrite without `--force`). With `--target` you can spin up an isolated copy for testing.

RESTORING ON TOP OF WORK

If you've made changes since the backup, `--force` overwrites them. Run `inkhaven backup` first if you want both versions on disk.

Recovery from drift

Three things can drift inside a project:

1. On-disk vs DB — you edited a `.typ` file outside inkhaven, the DB doesn't know. Fix: `inkhaven reindex` walks every `.typ`, re-reads it, updates the DB record.
2. DB vs vectors — DB knows about a paragraph but the vector store doesn't have its embedding (or has a stale one). Fix: `inkhaven reindex` also re-embeds.

3. DB vs metadata.db sanity — the DB itself has a broken record (extremely rare; usually power loss mid-write). Fix: `inkhaven reindex --prune` drops records pointing at missing files; `--adopt` adopts files the DB doesn't know about.

Chord	What it does
<code>inkhaven reindex</code>	Re-read every paragraph from disk, re-embed.
<code>inkhaven reindex --prune</code>	Also drop DB records whose file is missing.
<code>inkhaven reindex --adopt</code>	Add DB records for <code>.typ</code> files in the books tree that aren't tracked.

`inkhaven doctor`

A pre-flight health check:

```
inkhaven doctor
```

Reports the binary version, typst engine (external vs in-process), font availability, package cache state, project layout sanity, and any actionable warnings. Run this when something feels off.

```
inkhaven doctor - v1.2.6
=====
-- Binary --
version      1.2.6
description  Inkhaven - TUI literary work editor for Typst books
rust-version 1.85
repository   https://github.com/vulgov/blackInkhaven

-- Typst engine --
engine       external
external typst path /opt/homebrew/bin/typst
bundle fonts (K250M) true
use system fonts (K250M) true
packages enabled (K250M) true
semantic diagnostics (K250M) false

-- Package cache --
path         /Library/Caches/typst/packages
entries      688
size         5.8 MB

-- Project --
root         /testbook
status       initialised
config       /testbook/inkhaven.hjson
metadata.db  /testbook/metadata.db
vectors/     /testbook/vectors
books/       /testbook/books
user books   3
system books 9
paragraphs (total) 114
paragraphs (user) 108
words (user paragraphs) 142304

-- Notes --
no warnings
```

`inkhaven doctor` — health report. Green check + actionable warnings (yellow) + errors (red).

RECAP

- `inkhaven backup` writes a zip; `restore` reverses it.
- Auto-backup-on-exit triggered by `backup.max_age` (default 24h).
- `inkhaven reindex` heals on-disk ↔ DB ↔ vector drift.
- `--prune` drops dead DB records; `--adopt` claims orphaned files.
- `inkhaven doctor` is the pre-flight check before a real bug hunt.

12

Exporting your book

Export assembles every paragraph in a user book into one Typst document, then either ships it as-is (`typst` format), compiles it to PDF, or transforms it into Markdown / LaTeX / EPUB.

From the TUI

Chord	What it does
<code>Ctrl+B B</code>	Build PDF of the current user book. PDF lands in <code>`inkhaven-artefacts/<book-slug>/<book-slug>.pdf`</code> .
<code>Ctrl+B O</code>	Build extra formats listed in <code>`output.extra_formats`</code> .
<code>Ctrl+B A</code>	Assemble the typst-only source (no compile) into <code>`inkhaven-artefacts/<book-slug>/<book-slug>.typ`</code> .

`Ctrl+B B` opens a splash with a spinner while typst compiles. Esc cancels. When the engine is `inprocess` (the default since 1.2.5) the spinner stays inside the binary.

From the CLI

```
inkhaven export typst          # source-only
assembly
inkhaven export pdf            # full PDF
inkhaven export markdown      # paragraph→md
inkhaven export tex            # paragraph→latex
inkhaven export epub          # paragraph→epub3
```

Multi-book projects need `--book-name`:

```
inkhaven export pdf --book-name "Aerin Saga"
```

Filter flags

Two predicates ship in the CLI (1.2.4 + 1.2.6):

```
# Only paragraphs at Status:Final or above.
inkhaven export pdf --status final

# Only paragraphs carrying the `draft` tag.
inkhaven export pdf --tag draft

# Combined – both must pass (AND).
inkhaven export pdf --status final --tag draft
```

Status is the workflow ladder from Chapter 9; tag is the project-wide tag set from Chapter 14.

Configuration — `output.extra\_formats`

`Ctrl+B 0` walks the list:

```
output: {
  extra_formats: ["markdown", "epub"]
}
```

Each format produces its own artefact in `inkhaven-artefacts/<book-slug>/`.

Per-paragraph extraction

`Ctrl+V 1` (Editor scope) writes the OPEN paragraph as markdown to the current working directory. `Ctrl+V 2` does the same for the surrounding subchapter. `Ctrl+V 1` from the Tree pane variant writes the cursor's subtree.

Useful for quick “give me just this scene as markdown” exports without invoking the full pipeline.

EPUB metadata

EPUB writers want title + author at the very minimum. Inkhaven derives:

- Title — the book node’s title.
- Author — `output.epub_author` in HJSON, falling back to the git config’s `user.name`.

```
output: {  
  epub_author: "Vladimir Ulogov"  
}
```

Where everything lands

By default everything goes under `inkhaven-artefacts/<book-slug>/<book-slug>.<ext>` next to the project. Configurable:

```
artefacts_directory: ""    # empty → sibling  
                        # `inkhaven-artefacts/<project>/`  
`
```

The directory is gitignored in the default project template.

Render preview vs build

`Ctrl+V R` (Chapter 24) is paragraph-level render — quick, no surrounding context. `Ctrl+B B` is the full book — slower but correct. Use the preview for “does this paragraph look right?” and the build for “is this book shippable?”.

RECAP

- Ctrl+B B builds a PDF; Ctrl+B 0 walks `output.extra_formats`.
- CLI: `inkhaven export <typst|pdf|markdown|tex|epub>` with optional `--book-name`, `--status`, `--tag`.
- Per-paragraph quick export: Ctrl+V 1 / Ctrl+V 2.
- EPUB metadata: `output.epub_author` (falls back to git user).
- Artefacts land in `inkhaven-artefacts/<book>/` by default; configurable.

PART IV — WORLD BUILDING

13

Places and characters

Three system books — Places, Characters, and Artefacts — together

form what inkhaven calls the *lexicon*. Their titles power an inline highlight overlay in the editor: when you type a name that’s in the lexicon, it lights up.

Building entries

A character card is just a paragraph in the Characters book:

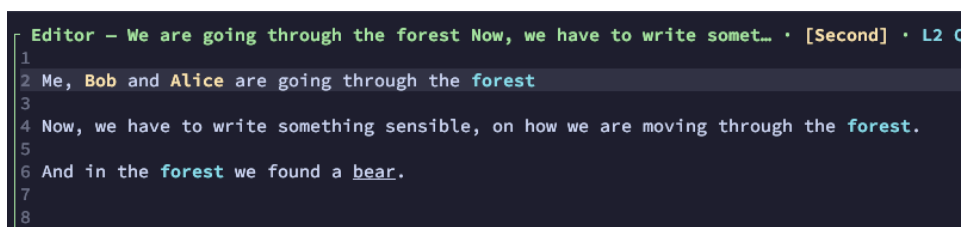
= Aerin Sandreaver

Twenty-three years old when the storm comes. Daughter of the Highkeep gatekeeper. Carries a wooden flute that belonged to her grandfather. Travels light; reads old maps in candlelight.

Same shape for Places and Artefacts. The title is what gets matched against your manuscript prose.

The highlight overlay

Open any paragraph. Type “Aerin walked into Highkeep” — both names light up:



```
Editor - We are going through the forest Now, we have to write somet... · [Second] · L2 C
1
2 Me, Bob and Alice are going through the forest
3
4 Now, we have to write something sensible, on how we are moving through the forest.
5
6 And in the forest we found a bear.
7
8
```

Lexicon overlay — character names in cyan, place names in yellow, artefacts in mauve. Subtle but always-visible.

Default colour scheme (Catppuccin Mocha):

Chord	What it does
Cyan	Character match.
Yellow	Place match.
Mauve	Artefact match.

Stemming

The lexicon scanner uses a Snowball stemmer keyed to your `language:` setting. So “Aerin” matches “Aerin’s”; “Highkeep” matches “Highkeep’s”. For Russian, “Москва” matches “Москве”, “Москвою”, etc. — declension-aware.

Stemmers available: english, russian, french, german, spanish, italian, portuguese, dutch, danish, swedish, norwegian, finnish, hungarian, romanian, arabic, basque, catalan, turkish, irish, tamil. Add `editor.stemming.languages` in HJSON for multiple at once.

RAG against an entry — `Ctrl+B P / C / A`

When the cursor is on a lexicon match, three chords ask the AI about the matched entry:

Chord	What it does
Ctrl+B P	Place RAG — sends the matched entry’s body + nearby paragraphs to the AI.
Ctrl+B C	Character RAG.
Ctrl+B A	Artefact RAG.

The matched entry lands as RAG context, and the AI pane prompt slot is focused so you can ask a follow-up question. Useful when you want to check whether your prose about Aerin matches her established card.

Lookup chords (no RAG)

Chord	What it does
Ctrl+B P (no match)	Open the Places book listing.
Ctrl+B C (no match)	Open the Characters book listing.
Ctrl+B A (no match)	Open the Artefacts book listing.
Ctrl+B N	Open the Notes book listing — same shape.
Ctrl+B U	Open the Research book listing.

When the cursor isn't on a match, the same chords open a list of every entry in the relevant book — pick one with Enter.

Image references in entries

Character cards can include images:

= Aerin Sandreaver

`#image("aerin-portrait.png", width: 60%)`

Twenty-three years old when the storm comes ...

Drop the PNG into `books/characters/`. The image renders in `Ctrl+V R` preview + the final PDF. Inkhaven's image node kind (Chapter 3) is a separate tree node you can reorder; the `#image()` call is independent of the node.

RECAP

- Lexicon = Places + Characters + Artefacts books. Titles auto-highlight in the editor.
- Snowball stemmer matches inflections; multilingual via `editor.stemming.languages`.

- Ctrl+B P/C/A — when on a match, RAG against the entry; when not, open the book listing.
- Ctrl+B N/U — Notes / Research book listings.
- Character cards can include images via `#image ( )` + co-located PNGs.

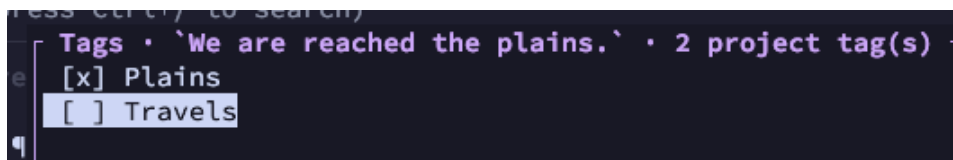
14

Tags

Tags are project-wide free-form labels you attach to paragraphs. They never embed in your manuscript text; they live as metadata. Use them for editing passes, beta- reader slices, POV tracking, plot threads — any cross-cutting concern that doesn't fit the tree.

Adding tags

Chord	What it does
Ctrl+B]	Open the tag picker for the open paragraph.
g (Tree)	Same picker for the tree-cursor's paragraph — works on multi-select.
Ctrl+B }	Project-wide tag search — every tag, every paragraph that uses it.



Tag picker — checkmarked tags are on the open paragraph. Space toggles; T commits; A adds new; R renames; D deletes.

Tags are case-preserved at write time but the picker dedupes case-insensitively. Tag namespaces (e.g. `pov-aerin`, `thread-revenge`) work fine — they're just strings.

Renaming tags project-wide

R inside the picker prompts for a new name and rewrites every paragraph carrying the old tag. If the new name already exists in the project, the rewrite merges — paragraphs that had both end up with just the destination.

Tags in the tree pane

Paragraph rows display their first two tags as compact `#tag` chips with `+N` for additional ones:



Tree paragraph rows with tag pips. `+N` shows when more than two tags are present.

Tag-filtered export

```
inkhaven export pdf --tag draft
inkhaven export pdf --tag draft --status final
```

The first writes a PDF with only paragraphs tagged `draft`. The second AND-combines with the status filter. Useful for beta-reader slices (“here’s the chapters I’ve marked beta-ready”) and submission packages.

Multi-select

Tags really shine when combined with tree multi-select:

Chord	What it does
Space (in tree)	Mark the cursor paragraph.
g (with marks)	Apply the tag picker to every marked paragraph at once.

So you can mark 12 paragraphs across three chapters and add the `pov-aerin` tag to all of them in one operation.

Bund — ``ink.tag.*`` `stdlib`

Five words plug tags into scripts:

```
ink.tag.list      ( -- list )
"intro/scene-1"  ink.tag.list_for ( path -- list |
NODATA )
"draft"          ink.tag.search  ( tag -- list-
of-paths )
"intro/scene-1" "draft" ink.tag.add    ( path tag -- )
"intro/scene-1" "draft" ink.tag.remove ( path tag -- )
```

Policy: reads under `store_read` (default allowed); writes under `store_write` (default denied — opt in).

Scrivener keyword import

`inkhaven import-scrivener` brings Scrivener's per-document keywords across as inkhaven tags. Both shapes are handled — the modern `<KeywordRef>`-against-registry form (Scrivener 3.x) and the older inline `<Keywords>foo, bar; baz</Keywords>` form. See Chapter 26.

RECAP

- Tags are project-wide free-form labels stored as metadata; never embed in prose.
- `Ctrl+B ] / g (tree)` — picker. `Ctrl+B }` — search.
- `R` inside the picker renames project-wide; merges into an existing tag if present.
- Tree pane shows first 2 tags as `#tag pips + +N`.
- `inkhaven export --tag <name>` filters export to a tag slice; combine with `--status`.
- Bund: `ink.tag.list / list_for / search / add / remove`.
- Scrivener keywords import automatically as tags.

15

Wiki-links and backlinks

Wiki-links are typed references between paragraphs. They live as metadata, never as embedded markers in the typst source. The link never leaks into your PDF; it sits in the paragraph’s `linked_paragraphs` field next to `status`, `tags`, etc.

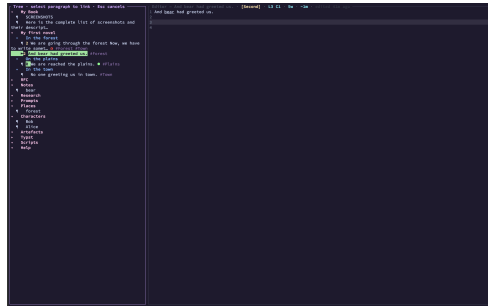
The metadata-only choice is deliberate: nothing about `[[X]]` syntax leaks into export, renaming a paragraph never breaks the link (UUIDs are stable), and the AI inference path can read structured references without parsing markup.

The four chords

Chord	What it does
Ctrl+V A	Outgoing — add a link FROM the open paragraph TO a picked paragraph.
Ctrl+V I	Incoming — add a link FROM a picked paragraph TO the open one (reverse of A).
Ctrl+V L	Open the linked-paragraphs picker (outgoing links).
Ctrl+V K	Open the backlinks picker (paragraphs linking TO the open one).

Adding a link

`Ctrl+V A` flips the tree pane into “select a target” mode. Navigate (arrows, slash-filter via `Ctrl+/,`) to the paragraph you want to link to, press Enter. The link lands; focus returns to the editor.



Tree in link-pick mode (Ctrl+V A). Title bar shows the purpose; Enter confirms; Esc cancels.

Guards

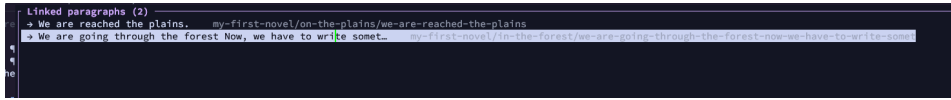
Three checks fire at link-add time:

Chord	What it does
Self-link	Rejected with `can't link a paragraph to itself`.
Duplicate	Reports `already linked`, does nothing.
Cycle	A DFS over the candidate target's outgoing closure looks for the owner. If found, rejected with `You can not create circular references`.

The cycle check makes outgoing-link metadata safe to walk — there's no infinite-recursion risk because there's no cycle. Walks AI-pass payloads, walks the timeline scope filter (Chapter 17), walks the story view edges (Chapter 16).

Listing + removing

Ctrl+V L lists outgoing; Ctrl+V K lists backlinks:



Linked-paragraphs picker. Each row shows direction (→) and slug-path. D removes; Enter opens.

Chord	What it does
↑ / ↓	Move cursor.
Enter	Open the linked paragraph.
D	Remove the link (just this one).
Esc	Close.

AI integration

When you ask the AI a question with the paragraph in scope, the AI sees (out: [list of titles]) and (in: [list of titles]) as RAG context. So “what does this scene set up?” gets a useful answer informed by which other scenes reference this one.

Story-view edges

Wiki-links appear as dashed edges in the Ctrl+V W book story view (Chapter 16). Visual auditing — see which paragraphs link out, which form hubs of reference, which are disconnected islands.

Bund — ``ink.event.link_paragraph``

For events (Chapter 17), `ink.event.link_paragraph` adds a link from an event to a paragraph. The same scrub on delete (1.2.6 AC, Chapter 14) walks `linked_paragraphs` AND `EventData.characters` / `places` — no stale link lingers when you delete its target.

RECAP

- Wiki-links are metadata, not source markup. They never appear in the PDF.
- `Ctrl+V A` adds outgoing; `Ctrl+V I` adds incoming; `Ctrl+V L` / `Ctrl+V K` list.
- Guards: self-link, duplicate, and cycle — refused at add time.
- Story view (Chapter 16) renders wiki-links as dashed edges.
- Delete-time scrub keeps links clean automatically.

16

The story view

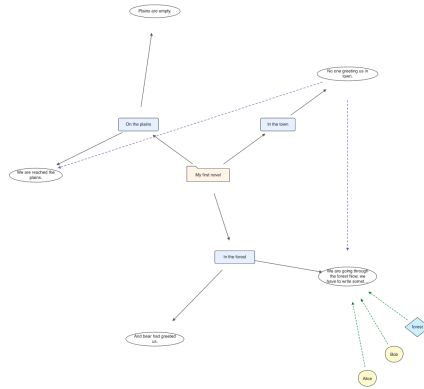
The story view is a radial graph of your book's topology — chapters, paragraphs, wiki-links, lexicon mentions — rendered as a PNG and floated over the editor. Pure visualisation: never mutates anything, just shows the shape.

The two chords

Chord	What it does
Ctrl+V Shift+W	Book view — book at the centre, everything on concentric rings.
Ctrl+V w	Paragraph mini view — open paragraph at the centre, hop-1 wiki-link neighbours + lexicon mentions on outer ring.

Case differentiates scope, mirroring the timeline chord (Chapter 17): lower-case = focus, uppercase = full read.

Book view



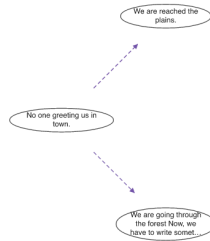
Ctrl+V Shift+W — book story view. Book at centre. Chapters, paragraphs, wiki-link dashed edges, lexicon mentions dotted edges.

Edge legend:

Chord	What it does
Solid	Hierarchy (parent → child).
Dashed	Wiki-link (Chapter 15).
Dotted	Lexicon mention (paragraph → matched Place / Character / Artefact).

Paragraph mini view

Ctrl+V w is the 20-node version — fast, focused, always readable. Inner ring: outgoing wiki-link targets on the right, incoming sources on the left. Outer ring: every Place / Character / Artefact whose title appears in the paragraph's body.



Ctrl+V w — paragraph mini view. Open paragraph at centre; hop-1 neighbours on inner ring; lexicon on outer.

Save the PNG

S inside either view opens a save-as picker pre-populated with `<book-slug>-story-YYYYMMDD-HHMM.png` (book) or `<paragraph-slug>-story-YYYYMMDD-HHMM.png` (paragraph). Enter writes; Esc cancels back to the view.

From Bund — ``ink.story.render``

Write the book view's PNG to a filesystem path without opening the modal:

```
"Aerin Saga" "~/Desktop/aerin-saga-story.png"
ink.story.render
```

Stack: (`book-name path --`). Case-insensitive book lookup against title + slug. `~/` expansion supported.

Policy: `fs_write` (default-denied — opt in via `scripting.enabled_categories`).

Useful as a `hook.on_save` side-effect that keeps a fresh graph on disk every time you save a paragraph — or as a nightly cron via a Bund script.

When the graph is too dense

Long projects produce dense graphs. A few patterns:

- Use the paragraph view (`Ctrl+V w`) — it's always readable.
- Run the book view per-book (`Ctrl+V Shift+W` respects the current book scope).
- Save and view externally — the PNG is full-DPI; open it in your image viewer for proper zoom.

RECAP

- `Ctrl+V Shift+W` is the book view; `Ctrl+V w` is the paragraph mini view.
- Three edge types: hierarchy (solid), wiki-link (dashed), lexicon mention (dotted).
- `S` saves to PNG; `Esc` closes.
- `ink.story.render Bund word` writes from a script (`fs_write` policy).
- Pure visualisation — never mutates.

PART V — THE TIMELINE

17

Story timeline

The timeline is inkhaven's first-class story-time metadata layer. Every scene-worthy paragraph can be tied to a calendar moment; the swim-lane view shows the book as horizontal tracks; AI critique audits the timeline for consistency. This is the headline feature of the 1.2.6 cycle.

The feature is opt-in: set `timeline.enabled: true` in HJSON to turn it on. Existing projects upgrade without surprise.

Enabling the timeline

```
timeline: {
  enabled: true
  default_track: "main"

  calendar: {
    preset: "gregorian"
  }
}
```

Three calendar presets:

Chord	What it does
`gregorian`	Real-world dates. `Y.M.D` format.
`sols`	Single-unit days since day zero. `Sol N` format.
`custom`	Anything else. Define units + seasons + format string yourself.

A custom calendar block for an Aerin Saga (12 months × 30 days, named months):

```
calendar: {
  preset: "custom"
  base_unit: "day"
  units: [
    { name: "day", names: [] }
  ]
}
```



```

    { name: "month", per_parent: 30,
      names: ["Frostmoon", "Snowfall", "Greenstart",
              "Bloomtide", "Highsun", "Goldfall",
              "Mistwane", "Stormrise", "Coldgate",
              "Longnight", "Hearthlit", "Yearfall"] }
    { name: "year", per_parent: 12, names: [] }
  ]
  seasons: [
    { name: "winter", start_month: 1, span_months: 3 }
    { name: "spring", start_month: 4, span_months: 3 }
    { name: "summer", start_month: 7, span_months: 3 }
    { name: "autumn", start_month: 10, span_months: 3 }
  ]
  epoch_label: "A"
  epoch_before_label: "BA"
  display_format: "{year}{epoch_label}.{month}.{day}"
}
```

Empty `names` falls back to numeric. Negative ticks (prequels) supported throughout.

Parser shapes

Chord	What it does
`1A`	Year — Precision::Year.
`1A.3`	Month — Precision::Month.
`1A.Frost`	Month-name prefix — also Precision::Month.
`1A.spring`	Season — Precision::Season (90-day window).
`1A.3.15`	Day — Precision::Day.
`-1BA`	Year before epoch (prequel) — Precision::Year.
`Founding`	Alias (defined in `parse_aliases`).

Precision matters for AI critique: a season-precision event collides differently against other season-precision events than against a day-precision one.

Where events live

The first time you add an event under a book — from the CLI or the TUI — a Timeline chapter materialises inside that book:

```
Aerin Saga/  
├─ Chapter 1/  
├─ Chapter 2/  
└─ Timeline/                                ← lazy; system_tag:  
  "book_timeline"  
    ├─ ◆ Birth of Aerin                    ← event paragraph (Node.event =  
Some(...))  
    ├─ ◆ Storm  
    └─ ◆ Marketplace scene
```

Each event is a paragraph carrying `EventData` in its metadata: `start_ticks`, optional `end_ticks`, `precision`, optional `track`, lists of `characters` / `places` / `linked_paragraphs`.

The Timeline chapter is auto-managed and *excluded from every export* — `inkhaven export pdf|markdown|tex|epub|typst`, the TUI's Ctrl+B B build, and the in-process render all skip both the Timeline chapter and any individual event paragraph (anything with `node.event.is_some()`). Your manuscript prose never gets timeline noise glued onto it.

Tree + editor visual cues

Chord	What it does
<code>`◆`</code>	Tree-pane glyph for an event paragraph (replaces the prose <code>¶</code>).
<code>`◆ <start>[ → <end>] · <prec> · <track>`</code>	Editor title bar shows full timing

Chord	What it does
<code>`[ORPHAN]`</code>	while editing an event. Red chip in the editor title when an event has no linked paragraphs / characters / places.
<code>`◆ linked from N event(s)`</code>	Editor title when a <code>#emph[manuscript]</code> paragraph is referenced by N events — same paragraph can anchor any number of events.

The visible timing in the editor means you can edit the event's body prose AND see at a glance which calendar moment it covers — no flipping to the swimlane view.

Adding events — three paths

Chord	What it does
CLI	<code>`inkhaven event add "Storm" --start "1A.2.3" --end "1A.2.5" --track main --book-name "Aerin Saga"``</code>
TUI · <code>`Ctrl+V Shift+E`</code>	From any pane: opens the timeline AND immediately fires the title prompt. Works on a fresh project — no chicken-and-egg with the CLI.
TUI · <code>`n` inside Ctrl+V Shift+T</code>	Position the timeline cursor first, then <code>`n`</code> for a title prompt

Chord	What it does
	at that tick + the highlighted track.

For the TUI paths, the title prompt commits with day-precision at the cursor’s current tick. To change the start, add an end, or rename the track, use the edit-timing chord (next section).

```

New event - n

New event @ Sol 15 · track: (default)
> Publish a new book|

Enter commits · Esc cancels

```

Ctrl+V Shift+E — title prompt for a brand-new event. The status bar shows the calendar-formatted tick the event will land on.

Editing event timing (Ctrl+V Shift+I)

When the open paragraph is an event, `Ctrl+V Shift+I` pops a one-line edit prompt for *start* | *end* | *track*, pipe-separated:

```

> Sol 13 | Sol 14 | main

```

Pre-filled with the event’s current values. Conventions:

Chord	What it does
<code>`Sol 13   Sol 14   main`</code>	Start, end, track.
<code>`Sol 13     main`</code>	No end (instant event).
<code>`Sol 13   Sol 14  `</code>	Drop the track (falls back to <code>`timeline.default_track`</code>).
<code>`Sol 13`</code>	Start only — no end, no track.

Enter commits all three at once. Precision is re-derived from the start string on each commit, so `Sol 13` is day-precision, `1A.3` is month-precision, etc. Bad parse fails to the status bar; the modal stays so you can fix it.

```
Edit event - Ctrl+V Shift+I
Edit event - format: start | end | track
> Sol 1 | | main|
Empty middle = no end · empty trailing = drop track · Enter commits · Esc ca
```

Ctrl+V Shift+I — edit prompt for the open event's timing. Pipe-separated start | end | track.

Linking events to manuscript paragraphs

An event is far more useful when anchored to the scene where it happens. To link from an open event paragraph:

Chord	What it does
<code>`Ctrl+V A`</code>	Enter link-pick mode: focus moves to the tree; navigate to the manuscript paragraph; <code>`Enter`</code> commits.
Bund	<code>`event-uuid "aerin-saga/ch4/marketplace" ink.event.link_paragraph`</code>

The link drops the `orphan` tag atomically with the link write — no separate “reconcile” step required. The same chord from the inverse direction (`Ctrl+V I` from the manuscript paragraph, picking the event in the tree) does the same write.

SAME PARAGRAPH, MANY EVENTS

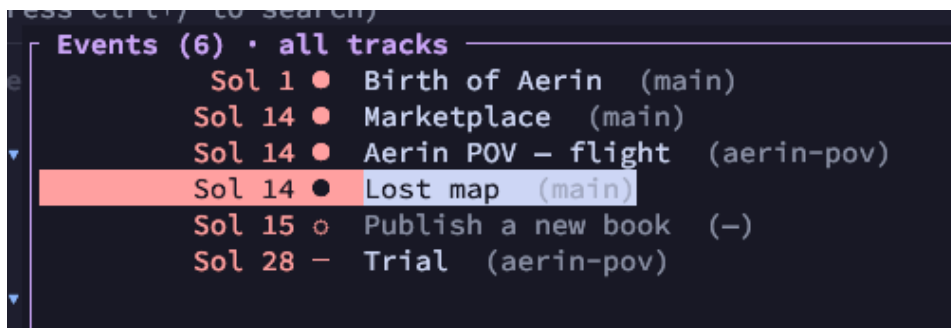
The data model is many-to-one: any number of events can carry the same manuscript paragraph in their `linked_paragraphs` list. Useful when a single scene resolves multiple plot lines, or when the same

setting recurs across POV tracks. The editor title bar surfaces the count (♦ linked from N event(s)) so the user knows a scene is timeline-anchored before opening Ctrl+V K. Removing a link reconciles the orphan tag on every affected event.

CLI

```
inkhaven event add "Storm" \  
  --start "1A.2.3" --end "1A.2.5" \  
  --track main --book-name "Aerin Saga"  
  
inkhaven event list --book-name "Aerin Saga" --track main  
  
inkhaven event show aerin-saga/timeline/storm
```

Ctrl+V e — vertical event picker



Ctrl+V e — chronological event picker. Track filter via `t`. Enter opens the event paragraph.

Chord	What it does
↑ / ↓ / Home / End	Move cursor.
t / T	Cycle track filter (None → t0 → t1 → ... → None).
Enter	Open the event paragraph in the editor.
Esc	Close.

Ctrl+V Shift+T — swim-lane view

The headline UI. Opens at the current paragraph’s nearest scope (Subchapter → Chapter → Book), auto-fitting the full event span (earliest start → latest end) inside the visible pane so you can see the whole timeline before drilling in:



*Ctrl+V Shift+T — swim-lane view, auto-fitted on open. Per-track rows: ● instant; — duration; ○ orphan.
Axis labels along the top.*

Four distinct navigation modes — fine scroll, event-hop, page-scroll, range-extremes:

Chord	What it does
← / →	Smooth scroll (~10 cells per press).

Chord	What it does
↑ / ↓	(1.2.6+) Hop the cursor to the previous / next event, chronologically. Viewport pans automatically to keep the new cursor on screen.
PgUp / PgDn	Page-scroll (~60 cells per press).
Home / End	Jump to first / last event.
+ / =	Zoom in (0.66× ticks/cell — anchored to cursor).
- / _	Zoom out (1.5× — anchored to cursor).
0	Reset zoom to 1.00×.
Tab	Cycle highlighted track.
Enter	Open the event closest to the cursor.
n / N	Pop a title prompt; commit a new event at cursor tick.

The auto-fit on open means you almost never need **0** (reset) manually — the initial frame already shows everything. Zoom preserves the cursor’s screen column through **+** / **-**, so drilling in feels like inspecting, not jumping.

When the book has *zero* events, the swim-lane view still opens — just empty, with a status hint pointing at **n** so the first event is one key away:

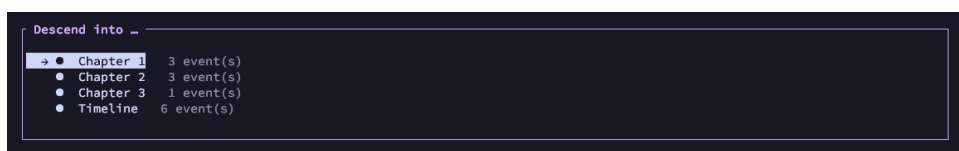


Ctrl+V Shift+T on a fresh book with no events. The status bar tells you ‘press n to add the first event’.

Scope navigation

Chord	What it does
u / U	Up-scope (subchapter → chapter → book).
d / D	Open the inline descent picker — immediate children with event counts.
b / B	Jump straight to book scope.
p / P	Toggle project overlay (every user book; tracks prefixed with book slug).

Event filter rule: at book scope all events show; at sub-book scope an event appears iff itself OR any of its `linked_paragraphs` is a descendant of the scope.



Descent picker (``d``) — immediate child scopes with their event counts. Enter descends; Esc returns.

AI health critique

Three chords kick off an AI consistency audit:

Chord	What it does
y	Current view scope, highlighted track only.
Y	Current view scope, all tracks.
Ctrl+Y	Book scope (widens regardless), all tracks.

The payload includes one bullet line per event with calendar-formatted start, optional end, title, track, precision, and `[ORPHAN]` tag where applicable.

Linked paragraphs resolve to slug-paths; characters / places resolve to titles. Trailing audit checklist instructs the model:

WHAT THE AI LOOKS FOR

- Travel-time / co-location conflicts.
- Paragraph references that contradict event dates.
- Fuzzy-precision overlaps (season events colliding).
- Orphan signals (events that look like they want a scene).
- Pacing — long gaps, rushed sequences.

Prompt resolves through the standard chain (Prompts book paragraph named `timeline-health` → `prompts.hjson` → embedded fallback). The `05-timeline-health-example.typ` seed lands in the Prompts book on `inkhaven init`.

Bund — ``ink.event.*``

Seven stdlib words:

```
ink.event.list      ( --
list )

ink.event.list_orphans ( --
list )
"Aerin Saga" "Storm" "1A.2.3"  ink.event.add      ( book
title spec -- uuid )
event-id "1A.2.5"      ink.event.set_end          ( uuid
spec -- )
event-id "season"      ink.event.set_precision    ( uuid
prec -- )
event-id "main"        ink.event.set_track        ( uuid
track -- )
event-id "aerin-saga/ch4/scene" ink.event.link_paragraph
                                ( uuid
path -- )
```

Policy: reads under `store_read`; mutations under `store_write`.

Bund hooks

Chord	What it does
<code>hook.on_event_added(uuid)</code>	Every successful add — CLI, TUI <code>`n`</code> chord, Bund <code>`ink.event.add`</code> .
<code>hook.on_event_orphaned(uuid)</code>	When an event transitions linked → orphan. Catches deletes via the AC scrub.

Orphans

An event with no `linked_paragraphs` AND no characters AND no places auto-tags `orphan`. The reconciler runs on every metadata write that touches an event — including link adds/removes via `Ctrl+V A` from the TUI, `Ctrl+V Shift+I` edits, and the Bund / CLI write paths. Orphans render with  across every UI (CLI list, picker, swim lanes) and the red `[ORPHAN]` chip in the editor title bar.

They're not errors — they're a soft signal that the event might want a scene attached.

To clear an orphan from the TUI: open the event paragraph (the editor title's hint reminds you), press `Ctrl+V A`, pick the manuscript paragraph in the tree, `Enter`. The write that adds the link removes the `orphan` tag in the same metadata update.

RECAP

- Opt-in: `timeline.enabled: true` + a calendar preset.
- Calendars: `sols`, `gregorian`, or `custom` (configurable named months, seasons, fuzz windows).

- Three add paths: CLI `event add`, TUI `Ctrl+V Shift+E` from anywhere, `n` inside the timeline.
- Edit timing: TUI `Ctrl+V Shift+I` on an event ¶ — pipe-separated `start | end | track`.
- Tree shows events with ♦; editor title shows ♦ `<timing>` for events and ♦ linked from `N event(s)` for anchor paragraphs.
- Multi-link: any number of events can target the same manuscript paragraph; orphan tag reconciles atomically with every link write.
- `Ctrl+V Shift+T` swim lanes auto-fit on open. Four nav modes: ↑↓ event-hop, ←→ scroll, `PgUp/PgDn` page, `Home/End` extremes.
- Scope nav: `u` up, `d` descent picker, `b` book, `p` project.
- AI critique: `y` track, `Y` scope, `Ctrl+Y` book-wide.
- Bund: 7 `ink.event.*` words + 2 hooks (`on_event_added`, `on_event_orphaned`).
- Timeline data is auto-excluded from every export format — your manuscript never picks up timeline noise.

18

Configuring AI providers

Six LLM providers ship inside the inkhaven binary via the `genai` crate. Configuration is one stanza in `inkhaven.hjson`; switching between providers at runtime is `Ctrl+B L`.

The full set

Chord	What it does
<code>gemini</code>	Google. Set <code>`GEMINI_API_KEY`</code> .
<code>claude</code>	Anthropic. Set <code>`ANTHROPIC_API_KEY`</code> .
<code>openai</code>	OpenAI. Set <code>`OPENAI_API_KEY`</code> .
<code>deepseek</code>	DeepSeek. Set <code>`DEEPSEEK_API_KEY`</code> .
<code>grok</code>	xAI Grok. Set <code>`XAI_API_KEY`</code> .
<code>ollama</code>	Local — no API key. Set <code>`host`</code> if not on localhost.

Pick one as default; the others stay configurable for `Ctrl+B L` live-switching.

Minimal configuration

```
llm: {
  default_provider: "ollama"

  ollama: {
    model: "qwen2.5:7b"
    host: "http://localhost:11434"
  }

  claude: {
    model: "claude-sonnet-4-6"
  }

  openai: {
```

```

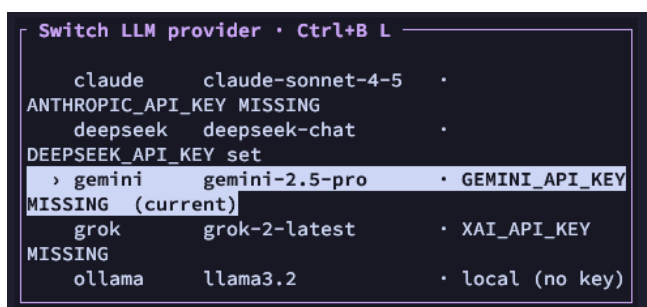
    model: "gpt-4o-mini"
  }
}

```

Inkhaven reads the API key from the matching environment variable. No keys in the HJSON, no keys in git. The config holds only the provider name + the model preference.

Live switching — Ctrl+B L

Ctrl+B L opens a small picker over every configured provider:



Ctrl+B L — provider picker. Current provider marked. Enter switches.

The switch is per-session; the default in HJSON survives a restart. Useful for picking the cheap fast model for grammar checks and the heavy model for critique.

Inference mode — Ctrl+B M / F10

Chord	What it does
Local	The model is constrained to the supplied RAG context. Won't draw on outside knowledge.
Full	The model can augment context with general knowledge (still treats context as ground truth).

F10 toggles the mode globally. **Ctrl+B M** shows the current state in the status bar. Help RAG (**F1**) is always pinned to Local — the help-answer flow refuses to confabulate features that don't exist.

AI scope — F9

F9 cycles the inference scope:

Chord	What it does
None	Just the user's typed query.
Selection	+ the current editor selection.
Paragraph	+ the whole open paragraph.
Subchapter	+ every paragraph in the current subchapter.
Chapter	+ every paragraph in the current chapter.
Book	+ every paragraph in the current book.

Cycle forward with **F9**; backward with **Shift+F9**. The current scope shows in the AI pane's title bar. Auto-resets to None after every send so you don't accidentally re-attach a huge context to your next casual question.

Cost discipline

The default mode for cloud providers is Full + Paragraph scope — substantial but not enormous. A typical 500-word-paragraph + 1500-token response is a fraction of a cent on every provider except Claude Opus.

If you're using a cloud provider heavily, set `ai.per_paragraph_memory_max_turns` to something modest (default 10 → consider 4-6) so paragraph-scoped chats don't grow unbounded.

Privacy

This deserves its own section. The five cloud providers (Gemini, Claude, OpenAI, DeepSeek, Grok) are not private. Every prompt you send + every RAG-attached paragraph travels to the provider's servers. They handle that data under their terms of service — which usually means they may log it, train future models on it, retain it under regulatory holds, or all three. The same applies to F1 Help RAG, F7 grammar check, F12 critique, Ctrl+F12 explain, and the timeline health critique — every one of these chords sends content to the configured provider.

Inkhaven gives you the controls — **F9** (scope), **F10** (Local vs Full inference mode), **Ctrl+B L** (provider switch), **Ctrl+B C** (clear chat history) — but the controls don't change the fact that something left your machine the moment you pressed the chord.

FOR INCREASED PRIVACY: OLLAMA

Set `llm.default_provider: "ollama"` and run a local Ollama instance (Linux / macOS: install via `brew install ollama` or the official Linux script; Windows: `winget install Ollama.Ollama`). Pull a model with `ollama pull qwen2.5:7b` (or any other model). At that point every inkhaven AI feature — F7 / F12 / Ctrl+F12 / F1 / timeline critique / chat / per-paragraph memory / suggest-event — runs on your machine. Nothing leaves the host except prose you've explicitly sent.

Inkhaven's other AI-adjacent subsystems are already on-device by design:

- RAG embedding (fastembed → ONNX runtime, no network)
- Semantic search (HNSW vector store, no network)
- Snapshot diff (pure Rust, no network)
- Spell/lex highlighting (Snowball, no network)

Ollama closes the loop on the LLM itself.

If you write entirely without AI, inkhaven works fully without a provider configured. Every non-AI feature in the book stands on its own. The chords just land helpful hints when there's no provider behind them.

Configuring per-provider

Each provider has its own block under `llm.<provider>:`. Common fields:

Chord	What it does
<code>model</code>	Which model name to use.
<code>host (ollama)</code>	URL of the Ollama API.
<code>base_url (openai)</code>	Override for OpenAI-compatible endpoints (e.g. local proxies).
<code>max_tokens</code>	Cap for the response length (default varies by provider).

The full reference is in Appendix B + `Documentation/CONFIGURATION.md`.

RECAP

- Six providers bundled: Gemini, Claude, OpenAI, DeepSeek, Grok, Ollama.
- API keys live in environment variables; HJSON only names the model + default provider.
- `Ctrl+B L` switches provider live (per session).
- `F10` toggles Local vs Full inference mode; `F9` cycles RAG scope.
- Five cloud providers offer no inherent privacy. For privacy, use Ollama — every other inkhaven subsystem is already on-device.

19

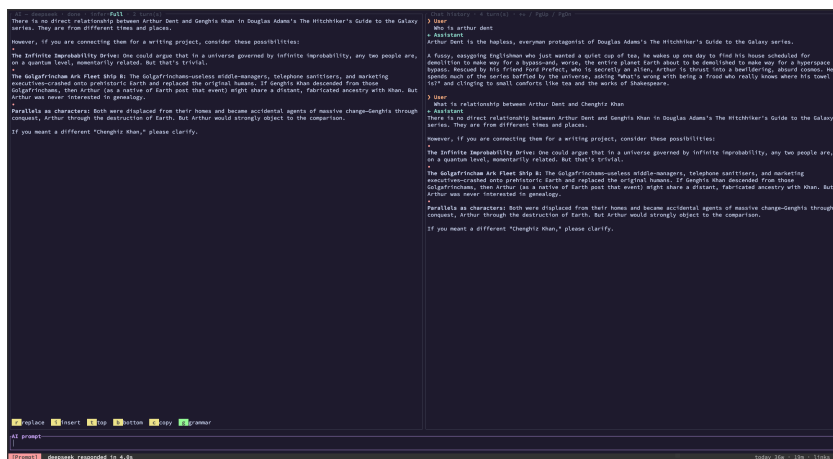
The AI pane

The AI pane is the right column of the three centre panes. It holds the chat history and the inference status; the prompt input sits at the bottom of the screen (**Ctrl+I** focuses it). A set of chords applies the AI's last response to the editor.

Two layouts

Chord	What it does
Default (3-pane centre)	AI pane is the right column of the three centre panes.
Ctrl+B K (full-screen)	AI pane fills the screen. Editor + tree hidden; the entire space is conversation.

Ctrl+B K is what you reach for when you want to think aloud with the model for a stretch and don't need the manuscript visible.



Full-screen AI pane (**Ctrl+B K**). Chat history fills the screen; prompt at the bottom.

Sending a prompt

Ctrl+I focuses the AI prompt slot. Type, Enter sends. While the stream is in flight you can keep typing — Inkhaven shows the response as it arrives.

Esc from any pane stops the stream cleanly (the partial response stays in history).

Scope (F9) recap

The scope you pick with F9 (Chapter 18) gets RAG-loaded as context BEFORE your typed query. So “explain this” with Paragraph scope sends the open paragraph + your query.

Applying answers

Five chords apply the AI’s last response to the editor:

Chord	What it does
r / R	Replace — overwrite the whole buffer (gated by the diff modal; see Chapter 21).
g / G	Replace with grammar-corrected text only (extracts the corrected block — see Chapter 20).
i / I	Insert at the cursor.
t / T	Prepend (top of the buffer).
b / B	Append (bottom).
c / C	Copy to clipboard (no edit).

r and **g** route through the diff-review modal by default (`ai.diff_review_on_apply: true`). **i** / **t** / **b** are additive and skip the modal.

Chat history

Inkhaven persists the chat history per project. Restart the TUI and the conversation is still there. Two chords manage it:

Chord	What it does
Ctrl+B C	Clear chat history — also clears the F7 grammar-change baseline.
Up arrow (prompt slot)	Walk backwards through your previous prompts.

Search the chat (`Ctrl+F` in AI pane)

Long chat histories are searchable:

Chord	What it does
Ctrl+F (AI pane focus)	Open the chat-search input.
Type	Filter to messages containing the query.
n / N	Walk hits.
Esc	Close — chat reverts to chronological.

Selection mode (`Ctrl+C` in AI pane)

A second mode that lets you pick a specific turn from history and either copy or insert it into the editor:

Chord	What it does
Ctrl+C (AI pane focus)	Enter selection mode.
↑ / ↓	Navigate turns.

Chord	What it does
c	Copy the selected turn to clipboard.
t	Insert the selected turn at editor cursor.
Esc	Exit selection mode.

Status messages

The status line under the AI pane reports the provider, the model, the inference mode (`Local` / `Full`), and the scope of the most recent send. Useful when you're mixing cloud + local providers and want to confirm where the next turn is going.

Quick prompts (Ctrl+B G — Notes RAG)

`Ctrl+B G` sends a query against the Notes book — semantic search across every note + the query packaged for the AI. Useful when you've scribbled something three weeks ago and can't remember where.

RECAP

- `Ctrl+I` focuses the prompt slot; Enter sends.
- `F9` cycles scope (None → Selection → Paragraph → ... → Book).
- Apply chords: `r` replace · `g` grammar replace · `i` insert · `t` top · `b` bottom · `c` copy.
- Chat history persists per project; `Ctrl+B C` clears it.
- `Ctrl+F` searches chat; `Ctrl+C` enters selection mode (copy / insert turns).
- `Ctrl+B K` goes full-screen for long-form AI sessions.

20

Prompts and the F7 grammar check

Every AI feature in inkhaven uses a *prompt template* — a piece of text that gets sent to the model before your prose. Templates resolve through a three-step chain so you can override anywhere from “I want a tiny tweak” to “I want a totally different prompt”.

The prompt resolution chain

For every named prompt (`grammar-check`, `critique-edit`, `timeline-health`, etc.):

Chord	What it does
1. Prompts book paragraph	Body of the paragraph whose title matches the name (case-insensitive).
2. <code>prompts.hjson</code> entry	Falls back to a per-project HJSON file.
3. Embedded default	The fallback compiled into the inkhaven binary.

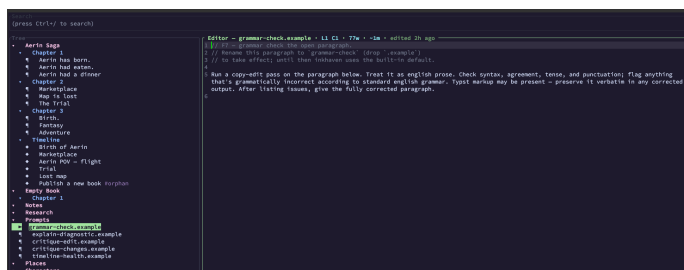
You override at any layer. The `.example` seeds in your Prompts book give you the embedded default verbatim, ready to customise.

The Prompts book seeds

`inkhaven init` writes five seeds:

```
Prompts/  
├─ grammar-check.example  
├─ explain-diagnostic.example  
├─ critique-edit.example  
├─ critique-changes.example  
└─ timeline-health.example
```

Open one with Enter from the tree pane. Edit the body. F2 to rename and drop the `.example` suffix. From that moment inkhaven uses your version.



Prompts book in the tree pane — five .example seeds. Rename (F2) drops the suffix to activate.

F7 — the grammar check

F7 (Editor scope) runs the grammar-check workflow:

1. Reads the open paragraph's body.
2. Resolves the `grammar-check` prompt.
3. Sends the prompt + paragraph body to the AI.
4. Focuses the AI pane so you can watch the stream.

The default prompt (English) is roughly: “Run a copy-edit pass. Check syntax, agreement, tense, punctuation. Preserve Typst markup. List issues then give the corrected paragraph between markers.”

`g` to apply (grammar-aware)

g (in AI pane) is the grammar-apply chord. Different from **r** (plain replace) because it extracts only the corrected paragraph from the response, ignoring the issue list and commentary.

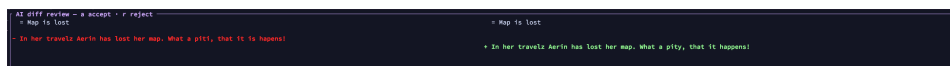
Extraction tries, in order:

1. `<<<CORRECTED>>>` / `<<<END>>>` markers (the canonical form the prompt instructs the model to produce).
2. Relaxed bracket pairs — `<<>>` / `<<END>>` / Unicode `«»` / `«>` (deepseek and other models drift to these compressed forms).
3. The last fenced code block.
4. Text after a `Corrected:` heading.

If none match, **g** refuses with a clear hint. **r** falls through to the markdown-conversion path on the full response.

Visual diff after apply

g paints the diff into the editor — added text in green, unchanged text plain. Saves automatically. The diff stays visible until you switch paragraphs (or **Ctrl+B C** clears it).



*After `g` — corrected paragraph in place, additions highlighted green. Survives saves; cleared by **Ctrl+B C**.*

Customising the grammar prompt

Open **Prompts/grammar-check.example** in the editor. The body is the default prompt. Rewrite it — common changes:

- Different language — replace “English” with your manuscript’s language. Stemmer + the AI both pick it up.
- Genre voice — add “preserve the present-tense literary style; flag any past-tense slips”.
- Stricter / looser — “ignore comma splices, this is a stylistic choice” or “flag every comma splice”.

F2 to rename: **grammar-check.example** → **grammar-check**. The next F7 picks up your version.

ROUND-TRIP AFTER EDITING

After renaming, test on a paragraph you know — F7, watch the AI use your prompt, g to apply. If the markers aren't right (model didn't emit <<<CORRECTED>>>), check the prompt — your override needs the marker instructions too.

Other prompt overrides

The same flow works for the other four seeds:

- **explain-diagnostic** — Ctrl+F12 (Chapter 24).
- **critique-edit** — F12 in plain edit mode (Chapter 21).
- **critique-changes** — F12 in split-edit mode.
- **timeline-health** — y/Y/Ctrl+Y in the timeline view (Chapter 17).

All five resolve through the same Prompts-book → HJSON → embedded chain.

RECAP

- Prompt resolution: Prompts book paragraph → prompts.hjson → embedded fallback.
- Five .example seeds land at init; rename (F2) to drop suffix and activate.
- F7 runs grammar check; g extracts only the corrected paragraph + applies.
- Visual diff after apply persists until paragraph switch or Ctrl+B C.
- Customise per-prompt by editing the seed body.

21

Critique, memory, and the diff modal

Three 1.2.6 additions work together: an F12 critique chord, opt-in per-paragraph chat memory, and a diff-review modal that gates buffer-replacing applies. In combination they make AI feel like a writing partner you trust rather than a fire-and-forget tool.

F12 — critique (mode-aware)

F12 (Editor scope) sends the open paragraph to the AI with a configurable critique prompt. The prompt depends on editor mode:

Chord	What it does
Plain edit	Prompt: `critique-edit`. AI sees the paragraph body.
Split-edit (F4)	Prompt: `critique-changes`. AI sees the snapshot body (Before) + the live buffer (After).

The default **critique-edit** prompt is roughly: “Point out the weakest two or three elements. Be specific — quote the phrase and propose a tighter alternative. Don’t rewrite the paragraph; critique it. Be honest, not destructive.”

critique-changes is roughly: “Identify what the revision changed. Evaluate each change — improvement, regression, or neutral. Suggest one focus for the next pass.”

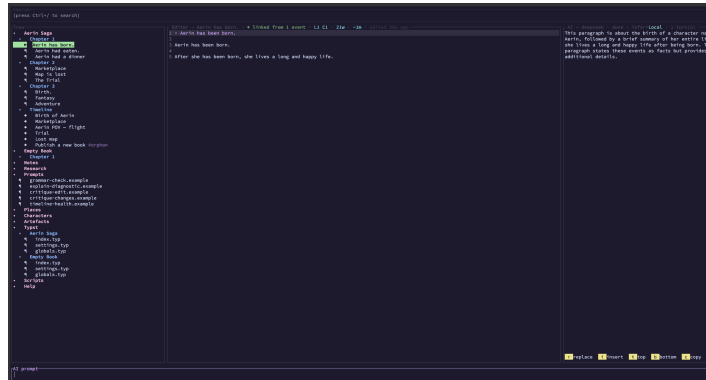
Per-paragraph AI memory

Opt in:

```
ai: {
  per_paragraph_memory: true
  per_paragraph_memory_max_turns: 10
}
```


When on AND F9 scope is Paragraph AND a paragraph is open, each AI turn records onto the paragraph's `Node.ai_memory` field. Subsequent Paragraph-scoped prompts prepend that memory to the chat history sent to the model.

Effect: a paragraph carries its own conversation with the AI. Come back days later, the model sees the prior context automatically.



Per-paragraph memory — three Paragraph-scope prompts on the same paragraph. The model sees turn 1 + turn 2 as prologue to turn 3.

The visible chat history stays project-wide and shows just the current session's turns. Memory is the invisible prologue.

Trimming is pair-aligned: when length exceeds `max_turns`, the oldest user+assistant pair evicts as a unit.

Recording only fires when the user's message wasn't empty. One-shots (F7 grammar, F12 critique, F1 Help, Ctrl+F12 explain) skip recording — they have no carrying-over story.

The diff modal — accept / reject

`ai.diff_review_on_apply: true` (default) routes every buffer-replacing apply through a side-by-side diff modal:

```

AI diff review - a: accept - r: reject
- Aerie has been born.
+ Aerie has been born.

- After she has been born, she lives a long and happy life.

+ Here is a critique of the paragraph, focusing on specific weak elements.
+ #1. Vague verb / dropped subject (production error as style):-
+ #quoted
+ "Aerie has born."
+ This is ungrammatical in English (should be "was born" or "has been born"). Even if treated as a stylistic choice, the use
+ "fighter alternative": "Aerie is born." (Simple present for an eternal, mythic fact; clean and immediate.)
+ }
+ #2. Abstract noun where concrete would land harder:-
+ #quoted
+ "After she has been born, she lives a long and happy life."
+ The phrase "long and happy life" is a generic summary. It tells the reader a conclusion rather than giving any sensory or
+ "fighter alternative": "After she is born, she grows up, falls in love once badly, plants a walnut tree, outlives two cats"
+ }
+ #3. Rhythm that flattens / redundancy:-
+ The entire paragraph repeats "has been born" / "has been born" / "has been born" in three consecutive sentences, then adds
+ "fighter alternative (not a rewrite, just a fix suggestion):- Cut the second and third sentences entirely. One clean, star
+ }

```

AI diff modal — left is current buffer, right is the proposed replacement. Removed lines marked -, added lines marked +.

Chord	What it does
a / A / Enter	Accept — apply the rewrite, refocus the editor.
r / R	Reject — close, buffer unchanged.
e / E	Accept + edit (alias for `a` since 1.2.6).
↑ / ↓ / PgUp / PgDn	Scroll the diff.
Home / End	Jump top / bottom.
Esc	Same as reject.

Modal status reports `> extracted X/Y chars` when the extraction layer trimmed commentary from the response. See Chapter 20 for the extraction logic.

Smart extraction — what lands in the buffer

For **r** and **g**, inkhaven scans the AI reply for a discrete corrected block before applying:

1. `<<<CORRECTED>>>` / `<<<END>>>` markers (canonical).
2. Relaxed bracket pairs (ASCII `<<>>`, Unicode `«»` / `⟨⟩`).
3. The last fenced code block.
4. Text after a `Corrected:` heading.

If any matches, only that block lands. If none match AND the action is `g`, the apply refuses. If none match AND the action is `r`, the full reply gets converted through markdown→typst and lands.

Disabling the diff modal

Some workflows feel slower with the modal. Turn it off:

```
ai: {
  diff_review_on_apply: false
}
```

`r` / `g` revert to immediate apply (the pre-1.2.6 behaviour).

Configuration recap

```
ai: {
  per_paragraph_memory:          false    # default
  per_paragraph_memory_max_turns: 10
  diff_review_on_apply:          true     # default
  reseed_prompt_examples:        true
}
```

The first two opt into stateful AI; the third gates the diff modal; the fourth controls whether the Prompts book gets re-seeded with `.example` files on every project open (idempotent — skips existing entries).

RECAP

- F12 critique — critique-edit in plain edit; critique-changes in split-edit.
- `ai.per_paragraph_memory: true` — paragraph-scoped chat continuity (prepend to history).

- `ai.diff_review_on_apply: true` (default) — side-by-side before `r / g` lands.
- Smart extraction picks the corrected block from the reply, trimming commentary.
- `a` accepts and refocuses editor; `r` rejects; `e` is an alias for `a`.

22

AI for diagnostics and the timeline

Two specialised AI surfaces sit at the intersection of writing and the technical layers: explain a Typst diagnostic in plain English, and audit a story timeline for consistency.

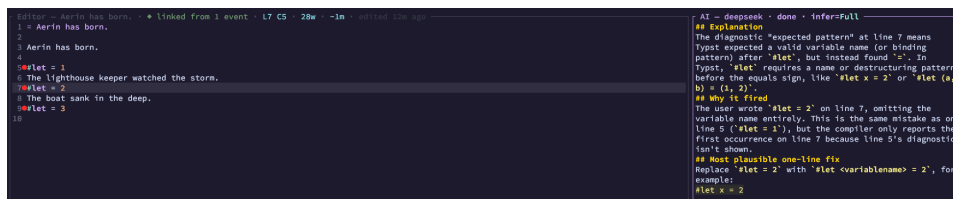
Ctrl+F12 — explain the diagnostic at cursor

When the gutter shows a red ● and you can't tell what the error means, Ctrl+F12 (Editor scope) bundles:

- the diagnostic message,
- the offending line + ±5 lines of context,
- the `explain-diagnostic` prompt template,

and streams the explanation into the AI pane.

The chord moved from F11 in 1.2.6 because macOS grabs F11 globally (Mission Control / Show Desktop). Linux + Windows users can rebind F11 back via the standard HJSON `keys.bindings` overlay.



The screenshot shows a Typst editor with a diagnostic message in the gutter. The diagnostic message is: "The diagnostic 'expected pattern' at line 7 means Typst expected a valid variable name (or binding pattern) after 'let', but instead found 'x'. In Typst, 'let' requires a name or destructuring pattern before the equals sign, like 'let x = 2' or 'let {a, b} = (1, 2)'." The AI pane on the right shows the explanation: "The diagnostic 'expected pattern' at line 7 means Typst expected a valid variable name (or binding pattern) after 'let', but instead found 'x'. In Typst, 'let' requires a name or destructuring pattern before the equals sign, like 'let x = 2' or 'let {a, b} = (1, 2)'." The AI pane also shows the most plausible one-line fix: "Replace 'let = 2' with 'let <variablename> = 2', for example: 'let x = 2'".

Ctrl+F12 — diagnostic message + ±5 lines of context sent to AI, which explains the cause + suggests a fix.

The default prompt is roughly: “A Typst compiler diagnostic is shown below with the surrounding source. Explain in plain English what the diagnostic means, why it likely fired in this context, and the most plausible one-line fix.”

Customising the explain prompt

Open `Prompts/explain-diagnostic.example`. Rename to `drop.example` once you’ve edited. Useful additions:

- “When the error references a custom function, check the book’s `globals.type` first.”
- “Be terse. Two sentences maximum.”
- “Always quote the offending phrase.”

Timeline health critique (y / Y / Ctrl+Y)

Inside the swim-lane view (`Ctrl+V t`):

Chord	What it does
y	Critique events in the current view scope, current track only.
Y	Critique events in the current view scope, all tracks.
Ctrl+Y	Critique events in the whole book, all tracks (widens regardless of view scope).

The payload — covered fully in Chapter 17 — is a flat prose summary the model can read without parsing calendar config. Events appear as bullet lines with calendar-formatted start, optional end, title, track, precision, orphan flag, and resolved character / place / paragraph link names.

What the audit asks for

The default `timeline-health` prompt instructs the model to surface:

- Travel-time conflicts — a character at two events with insufficient time between them.
- Paragraph mismatches — a manuscript paragraph saying “the day after the Storm” while the Storm is three months earlier.

- Fuzzy-precision overlaps — two season-precision events whose 90-day windows collide suspiciously.
- Orphan signals — events tagged orphan that look like they want a scene attached.
- Pacing — long unexplained gaps or rushed sequences.

Hook on diagnostic (Bund)

The diagnostic surface also fires a Bund hook when the state changes:

```
"hook.on_diagnostic" {
  // ( uuid count first-message -- )
  swap drop swap drop      // ( count )
  dup 5 >                  // ( count bool )
  { "⚠ many diagnostics: " print println }
  { drop } ifelse
} register
```

Debounced — fires only on state transitions (clean → errored, count change, top-message change). Doesn't fire on every idle tick when nothing moved.

`ink.editor.set\_cursor` for auto-jump

Pairs with the diagnostic hook for automated cursor positioning:

```
"hook.on_diagnostic" {
  // ... parse the message → get line + col ...
  drop drop drop
  3 42 ink.editor.set_cursor    // ( row col -- )
} register
```

1-based row/col (matches the diagnostic format). Policy: `editor_write` (default-allowed).

RECAP

- `Ctrl+F12` — AI explain for the diagnostic at cursor.
- Custom prompt: `Prompts/explain-diagnostic` (rename from `.example`).
- `y / Y / Ctrl+Y` inside `Ctrl+V t` — timeline health critique.
- `hook.on_diagnostic` — Bund reaction to diagnostic state changes.
- `ink.editor.set_cursor` — 1-based cursor mover for hook-driven scripts.

23

Typst inside Inkhaven

Inkhaven's typesetter is Typst — a modern, academic-friendly language

that compiles markup to beautifully-typeset PDFs. The compiler lives *inside* the inkhaven binary; you don't need to install Typst separately.

Two engines

Two compile paths ship in 1.2.5+:

Chord	What it does
<code>`engine: "external"`</code>	Default. Shells out to a <code>`typst`</code> binary on <code>`PATH`</code> .
<code>`engine: "inprocess"`</code>	1.2.5+. The full Typst compiler + fonts + <code>`@preview`</code> packages linked into the inkhaven binary.

Set in HJSON:

```
typst_compile: {  
  engine: "inprocess"  
  
  bundle_fonts:    true      # Bundle CM + Linux Libertine.  
  use_system_fonts: true      # Also walk host fonts.  
  packages_enabled: true      # Allow @preview/<pkg>  
  resolution.  
}
```

Why the in-process engine matters

Most TUI features that touch typst (render preview, semantic diagnostics, per-page navigation) require the in-process engine. The external CLI engine is faster to launch but can't expose the structured `SourceDiagnostic` data the in-process compiler returns.

If you ever see “this feature needs `engine = \"inprocess\"`” in the status bar, that’s why.

Per-book Typst skeleton

Every user book gets three auto-generated typst files in `books/<book-slug>/:`

Chord	What it does
<code>index.typ</code>	The entry point. Inkhaven appends every paragraph’s body to this.
<code>settings.typ</code>	Document-wide <code>`#set`</code> / <code>`#show`</code> rules. Edit to customise.
<code>globals.typ</code>	Functions / variables imported everywhere. Edit for custom typography.

Edit them in any text editor. Inkhaven re-reads them on every build / render. The default `globals.typ` is empty; the default `settings.typ` sets page size + font + paragraph indent from `typst_page` / `typst_fonts` / `typst_layout` in HJSON.

``@preview`` packages

Typst’s package ecosystem (`@preview/...`) works when `packages_enabled: true`. First reference downloads the package + caches it; subsequent references are offline.

```
#import "@preview/codly:1.0.0": *  
  
#show: codly-init.with()  
  
#raw(  
  "fn main() { println!(\"hello\"); }",  
  lang: "rust",  
)
```

Cache lives in `~/ .cache/typst/packages/`. Inkhaven respects typst’s existing locations so packages downloaded by the external CLI are reused by the in-process engine.

Diagnostics from Typst

Two checks run on the open paragraph (Chapter 24 covers the full surface):

Chord	What it does
Parse	`typst-syntax` only. Always on. Catches syntax errors.
Semantic	Full compile via the in-process engine. Catches reference errors, type errors, missing imports.

The parse check is engine-independent. The semantic check requires `engine: \"inprocess\"`.

Doctor — check the setup

```
inkhaven doctor
```

Reports engine choice, fonts available, package cache state, and any actionable warnings. Run after changing engines or moving the project.

```
— Typst engine —
engine           external · /opt/homebrew/bin/typst
external typst path  /opt/homebrew/bin/typst
bundle_fonts (HJSON) true
use_system_fonts (HJSON) true
packages_enabled (HJSON) true
semantic_diagnostics (HJSON) false
```

``inkhaven doctor`` — *typst* section. Engine, fonts (system + bundled), package cache, and any warnings.

Font bundling

When `bundle_fonts: true` (default), the binary ships Computer Modern + Linux Libertine inside it. A headless CI machine without system fonts can still build the book. Adds 10 MB to the binary.

For book-specific custom fonts, drop the `.ttf` / `.otf` files into `books/<book-slug>/fonts/` and add a `#set text(font: ...)` rule in `settings.typ`. Inkhaven's in-process engine sees them.

FONTS NOT FOUND?

Two common causes: (1) `bundle_fonts: false` AND `use_system_fonts: false` — no fonts at all. (2) The font name in `#set text(font: ...)` doesn't match the installed font's family name. `fc-list` (Linux) or Font Book (macOS) is the truth-source.

RECAP

- Two engines: `external` (default) shells out; `inprocess` (1.2.5+) is built-in.
- Most TUI typst features need `inprocess`.
- Per-book skeleton: `index.typ`, `settings.typ`, `globals.typ`.
- `@preview/<pkg>` works when `packages_enabled: true`.
- `inkhaven doctor` reports the setup; bundled fonts ship in the binary.

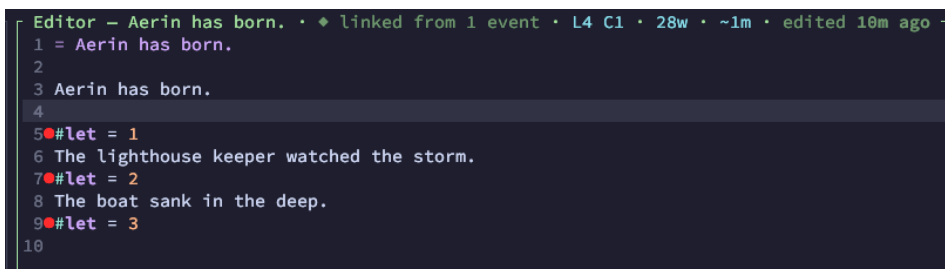
24

Diagnostics and render pre-view

Two features bring the Typst feedback loop into the editor: structured diagnostics surfaced at the line they fire, and a paragraph-level render preview that rasterises the open paragraph without running the full build.

Gutter markers

Lines that carry a diagnostic get a red ● in the line- number gutter:

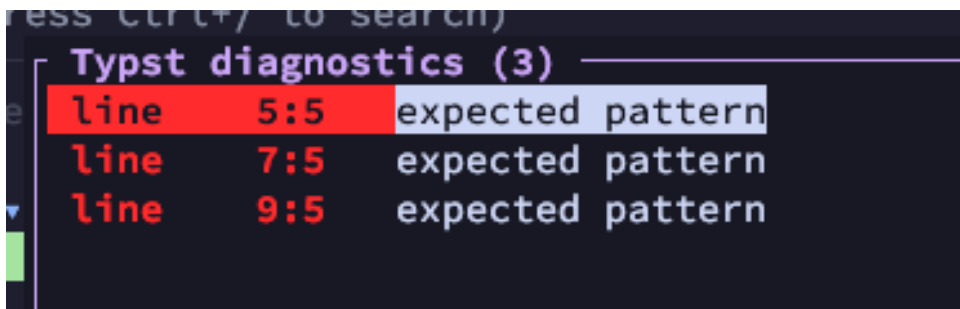
A screenshot of a code editor window titled "Editor - Aerin has born. • ♦ linked from 1 event • L4 C1 • 28w • ~1m • edited 10m ago". The editor shows a paragraph of text with line numbers 1 through 10 in the left margin. Line 1: "1 = Aerin has born." Line 2: "2" Line 3: "3 Aerin has born." Line 4: "4" Line 5: "5●#let = 1" Line 6: "6 The lighthouse keeper watched the storm." Line 7: "7●#let = 2" Line 8: "8 The boat sank in the deep." Line 9: "9●#let = 3" Line 10: "10" Red circular markers are visible in the gutter next to lines 5, 7, and 9. The text on line 5 is "#let = 1", on line 7 is "#let = 2", and on line 9 is "#let = 3".

Editor gutter — line 3 has a red ● because it references an undefined function. Marker stays visible while you fix.

Both parse + semantic diagnostics surface here. The marker slot is otherwise a space, so a clean buffer pays no visual cost.

F8 — diagnostics list

F8 (Editor scope) pops a list of every diagnostic in the open paragraph:



F8 — diagnostics list. Each row shows line:col + message. Enter jumps cursor; Esc closes.

Chord	What it does
↑ / ↓	Move cursor.
Enter	Jump editor cursor to the selected diagnostic; close the modal.
Esc	Close.

Ctrl+V N / Shift+N — next / previous diagnostic

Without opening the modal:

Chord	What it does
Ctrl+V N	Jump to next diagnostic in the open buffer (wraps).
Ctrl+V Shift+N	Previous (wraps).

Both refresh the diagnostics cache up-front so navigation follows the live buffer, not the last save.

Ctrl+F12 — AI explain

Covered in Chapter 22. Brief: AI explains the diagnostic at cursor with ± 5 lines of context.

Ctrl+V R — render preview

Quick paragraph-level render. Saves the paragraph first (so the render reflects on-disk state), then rasterises every page at 144 dpi:

Here is a critique of the paragraph, focusing on specific weak elements.

1. **Vague verb / dropped subject (production error as style)**
"Aerin has been"
This is ungrammatical in English (should be was here or has been here). Even if treated as a stylistic choice, the missing here makes the sentence feel like a fragment, not a deliberate omission. It confuses before it can lead.
Tighter alternative: "Aerin is here" (Strongly present for an eternal, mythic fact; clean and immediate.)

2. **Abstract noun where concrete would land harder**
"After she has been here, she lives a long and happy life."
The phrase long and happy life is a generic summary. It tells the reader a conclusion rather than giving any sensory or specific detail. The words long and happy are abstractions—empty adjectives.
Tighter alternative: "After she is here, she grows up, falls in love once, builds a modest home, nurtures two cats, and dies at ninety-one with honey on her lips" (Replace the abstraction with a few concrete, vivid markers.)

3. **Rhythm that flattens / redundancy**
The entire paragraph repeats has been here / has been here in three consecutive sentences, then adds a vague portrait. The repetition isn't rhythmic or necessary — it sounds like a technical record or a type loop. The third sentence ("After she has been here,") adds zero new information and actually diminishes the weight of the first declarative.
Tighter alternative (one sentence, just a 3x suggestion): Cut the second and third sentences entirely. One clean, stark line—"Aerin is here"—is stronger on its own. If you want a second sentence, make it specific and surprising, not a redundant echo.

Ctrl+V R — rendered PNG of the open paragraph. ← / → navigate pages, + / - zoom, S saves current page.

Chord	What it does
← / →	Previous / next page.
Home / End	First / last page.
+ / =	Zoom in (multiply ticks/cell by 0.66).
- / _	Zoom out (1.5×).
0	Reset zoom to 1.00×.
S	Save the current page to PNG.
A	Save every page to PNGs.
Esc	Close the modal.

Zoom range [0.05, 6.00]× the default DPI. Cursor’s screen column stays anchored through zoom — zooming feels like drilling in.

Save flows

S opens a save-as picker pre-populated with `<paragraph-slug>-YYYYMMDD-HHMM.png`. **A** writes one PNG per page with `-page-NNN.png` suffix.

Save always uses the default render DPI (not the zoom value) — zoom is for screen preview; save is for the artefact.

When NOT to use render preview

- Full-book layout — Use **Ctrl+B B** (build) and open the PDF in a viewer. Per-paragraph render strips cross-paragraph context (headers, page numbering).
- Mid-sentence rewrite loops — the path saves first. If you're holding many small edits, save manually with **Ctrl+S** and preview in batches.

Configuration

```
images: {
  preview_enabled: true      # required for Ctrl+V R
}

typst_compile: {
  engine: "inprocess"        # required for render
  diagnostics_idle_seconds: 2.0
  diagnostics_max: 50
}
```

The `images.preview_enabled` switch matters because on terminals without graphics support the half-block fallback is too coarse to be useful — false makes the chord land a clean hint instead.

`hook.on\_diagnostic` recap

For full coverage see Chapter 22. The hook fires only on state changes (not every idle tick) — `( uuid count first-message -- )`.

RECAP

- Gutter ● on every line with a diagnostic.
- F8 lists diagnostics; Enter jumps cursor.
- Ctrl+V N / Shift+N navigate in place; Ctrl+F12 AI explain.
- Ctrl+V R previews the open paragraph; +/- live zoom; S / A save PNGs.
- `hook.on_diagnostic` for Bund-side reactions.

25

Multi-format export

Export beyond PDF: the inkhaven binary ships converters for Markdown, LaTeX, and EPUB. The same filter flags (`--status`, `--tag`, `--book-name`) work across all formats.

The five formats

Chord	What it does
typst	Source-only assembly. The combined .typ file inkhaven would feed to the compiler.
pdf	Full PDF via typst-pdf.
markdown	Typst → Markdown. CommonMark-ish output, retains code blocks + math.
tex	Typst → LaTeX via the bundled `tylax` crate. Useful for academic submission.
epub	Typst → Markdown → EPUB3. The bundle uses your title + author metadata.

From the CLI

```
inkhaven export typst                # source-only
inkhaven export pdf
inkhaven export markdown
inkhaven export tex
inkhaven export epub

# Multi-book project – pick one book.
inkhaven export pdf --book-name "Aerin Saga"

# Filter: only Status:Final or above.
inkhaven export pdf --status final
```



```
# Filter: only paragraphs tagged `draft`.
inkhaven export pdf --tag draft

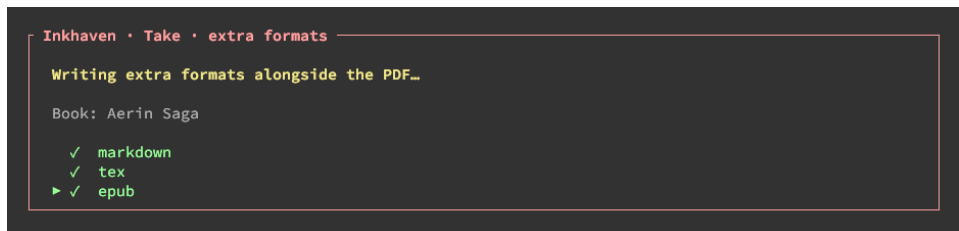
# Combined – AND.
inkhaven export pdf --status final --tag draft
```

Ctrl+B O — extra formats from inside the TUI

Configure your common formats once:

```
output: {
  extra_formats: ["markdown", "epub", "tex"]
}
```

Ctrl+B O walks the list and writes each artefact to `inkhaven-artefacts/<book-slug>/`. Useful as a “publish my draft” one-keystroke loop.



Ctrl+B O — splash showing each format being built, one at a time. Esc cancels (already-built formats survive).

Per-paragraph quick extracts (‘Ctrl+V 1/2’)

`Ctrl+V 1` (Editor scope) writes the OPEN paragraph as markdown to the current working directory. `Ctrl+V 2` does the same for the surrounding subchapter. `Ctrl+V 1` from the Tree pane writes the cursor's subtree.

Useful for “give me just this scene as markdown” without going through the full pipeline.

EPUB notes

EPUB readers need a title + author at minimum:

```
output: {  
  epub_author: "Vladimir Ulogov"  
}
```

Falls back to `git config user.name` when unset. The book title comes from the user book's title.

Cover image: if `books/<book-slug>/cover.png` exists, it gets used as the EPUB cover. Otherwise the EPUB ships coverless.

LaTeX notes

LaTeX export uses `tylax` — a pure-Rust Typst→LaTeX converter. Most prose round-trips cleanly; complex Typst constructs (custom math environments, advanced layout functions) translate to escape hatches and may need hand editing.

The `tex` output is one file: `<book-slug>.tex`. Useful for academic journals that require LaTeX submission.

Markdown round-trips

The Typst→Markdown converter handles:

- Headings (= H → # H)
- Bold / italic (*b* / i → **b** / *i*)
- Lists (numbered + bulleted)
- Code blocks (preserved)
- Footnotes
- Links

Things it doesn't preserve: typst-specific layout (margins, breaks), custom function calls, math typesetting (simplified to `$ \text{like this} $`).

Filter combinations

The two predicates compose:

```
# Only Status:Ready + tagged `submission` + the Aerin Saga
book.
inkhaven export pdf \
  --book-name "Aerin Saga" \
  --status ready \
  --tag submission
```

A paragraph must pass ALL active predicates. Useful for shipping precisely-scoped slices to specific readers.

RECAP

- `inkhaven export <typst|pdf|markdown|tex|epub>` — five formats.
- Filters: `--book-name`, `--status`, `--tag` — combine with AND.
- `Ctrl+B 0` walks `output.extra_formats` in HJSON.
- `Ctrl+V 1` / `Ctrl+V 2` — per-paragraph / per-subchapter markdown extracts.
- EPUB needs `output.epub_author`; falls back to git config.

26

Importing existing work

Most writers arrive at inkhaven with manuscripts already underway

— a markdown folder, a Scrivener project, a directory of Typst files. Three importers ship with the binary; all are CLI commands that produce a normalised inkhaven tree.

Markdown / Typst directory tree

`inkhaven import-help` was originally for populating the Help book but works for any directory of `.md` / `.typ` files. Subdirectories become chapters; files become paragraphs:

```
inkhaven import-help \  
  --documents-directory ~/Documents/old-book/
```

A directory like:

```
old-book/  
├─ chapter-01/  
│   ├── opening.md  
│   ├── conflict.md  
│   └─ resolution.md  
└─ chapter-02/  
    ├── new-day.md  
    └─ crisis.md
```

becomes:

```
Help/  
├─ chapter-01/  
│   ├── opening  
│   ├── conflict  
│   └─ resolution  
└─ chapter-02/  
    ├── new-day  
    └─ crisis
```

The target is the Help book by default (the importer's name gives this away) — re-purpose later with `inkhaven mv` if you want it in a user book.

The F3 file picker

For one-off imports inside the TUI:

Chord	What it does
F3 (Tree pane)	Open the file picker — navigate to a directory or .md/.typ file.
Enter (file)	Import as a paragraph at the cursor.
Enter (directory)	Import the tree under cursor.

Useful when you've got a single file you want to drop into the current chapter without leaving the TUI.

Scrivener import

`inkhaven import-scrivener` walks a Scrivener `.scriv` package, parses the binder XML, converts every RTF body to Typst markup, and materialises the hierarchy:

```
inkhaven import-scrivener \  
  --source ~/Documents/MyBook.scriv  
  
# Dry-run – preview without writing.  
inkhaven import-scrivener --source ... --dry-run
```

Single-binary — no Scrivener / pandoc / textutil required. Walks the binder structure, maps it to inkhaven nodes per the rules:

Chord	What it does
DraftFolder	Becomes a user Book.

Chord	What it does
Folder (under Draft)	Becomes a Chapter; nested becomes Subchapter (max one level).
Text	Becomes a Paragraph with the converted body.
Characters / Places / Notes folders	Mapped to the matching inkhaven system books.
Research / Notes top-level	Imported into the Research / Notes system books.

Scrivener keywords → tags (1.2.6+)

Scrivener’s per-document keywords come across as inkhaven tags. Both shapes handled:

- Modern Scrivener 3.x — project-level `<Keywords>` registry + per-item `<KeywordRef ID="N"/>` references.
- Older / lighter exports — inline `<MetaData><Keywords>foo, bar; baz</Keywords></MetaData>` with comma / semicolon / newline separators.

Both end up on `Node.tags` after import — source order preserved, case kept, duplicates dropped. Scope: paragraphs only (Scrivener allows keywords on folders too, but inkhaven’s tag picker is paragraph-focused).

Scrivener — what's NOT imported

- Snapshots — Scrivener’s snapshot history doesn’t cross over. The first paragraph version becomes the only one.
- Section types — Scrivener “section types” don’t map to inkhaven’s status ladder; everything imports as `Status:None`. Run `Ctrl+B R` to set rungs after.

- Compile groups — same story; use tags + `--tag` filter on export to recover the slicing.
- Custom metadata — `<CustomMeta>` blocks are dropped for now (1.2.8+ may bring a configurable mapping).

Typst help reference

Inkhaven ships with a curated Typst reference that you can opt into:

```
inkhaven import-typst-help
```

Creates a `Typst reference` chapter in the Help book. F1 RAG against the Help book now answers typst questions from grounded context — useful when you can't remember the syntax for some `#let` form or how to call `#image()`.

Importing the Book of Inkhaven into Help

Once you've got the markdown mirror of this book (under `Book/markdown/`), import it into the Help book:

```
inkhaven import-help \  
  --documents-directory ./Book/markdown/
```

F1 now answers questions about inkhaven itself from this book. End-state docs become end-state RAG.

RECAP

- Three importers: directory tree (`import-help`), Scrivener (`import-scrivener`), bundled Typst help (`import-typst-help`).
- F3 in Tree pane opens an inline file picker for one-off imports.
- Scrivener keywords import as inkhaven tags automatically (1.2.6+).
- Snapshots, section types, and custom metadata don't cross over.
- Import the book's markdown mirror into Help so F1 covers it.

27

Theming and the cheat sheet

Every colour in the inkhaven TUI is configurable. The defaults ship

Catppuccin Mocha — a popular warm-dark palette — but every knob is exposed in `inkhaven.hjson`.

The theme stanza

```
theme: {
  bg:          "#1e1e2e"      # base background
  fg:          "#cdd6f4"      # default text
  current_line_bg: "#313244"
  selection_bg:  "#414559"
  status_bar_bg: "#181825"
  modal_border:  "#cba6f7"
  # ... many more
}
```

Hex strings with the `#` prefix. Inkhaven applies them at boot. The full list is in Appendix B.

Quick palette swap

To switch the whole TUI to Gruvbox Dark:

```
theme: {
  bg:          "#282828"
  fg:          "#ebdbb2"
  current_line_bg: "#3c3836"
  selection_bg:  "#504945"
  modal_border:  "#fb4934"
  # ... customise to taste
}
```

To switch to Solarised Light:

```
theme: {
  bg: "#fdf6e3"
  fg: "#586e75"
  # ... light-theme adjustments
}
```

Per-syntax colours

The editor highlights typst syntax — headings, function calls, code blocks. Theme keys for those:

Chord	What it does
<code>syntax_heading</code>	Heading text.
<code>syntax_emphasis</code> / <code>syntax_strong</code>	Italic / bold text.
<code>syntax_function</code>	<code>`#function(...)`</code> calls.
<code>syntax_operator</code>	Operators.
<code>syntax_list_marker</code>	<code>`-`</code> / <code>`+`</code> markers.
<code>syntax_raw</code>	Inline + block code.
<code>syntax_quote</code>	Block quotes.

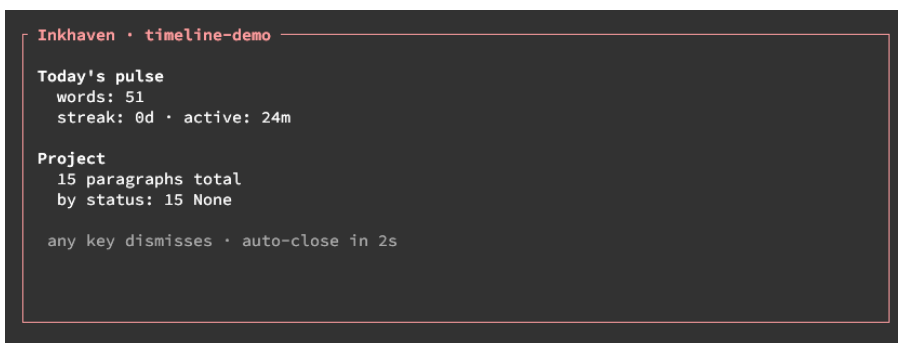
Lexicon overlay colours

Character / Place / Artefact highlights:

Chord	What it does
<code>editor_character_fg</code>	Character match (default cyan).
<code>editor_place_fg</code>	Place match (default yellow).
<code>editor_artefact_fg</code>	Artefact match (default mauve).

Editor knobs (other)

```
editor: {  
  typewriter_sounds:      false    # tap/return sound  
  effects  
    autosave_idle_seconds: 30  
    word_wrap:             true  
    splash:                 true    # show progress-pulse on  
  launch  
    paragraph_target_default: 250  
}
```

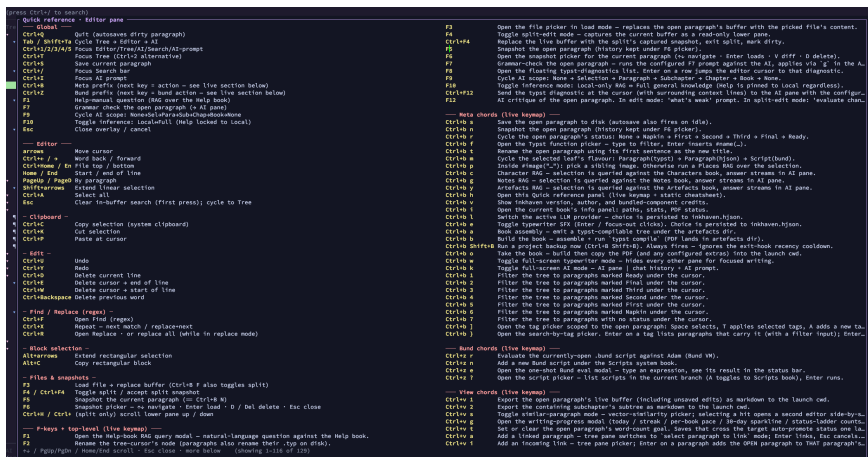


```
Inkhaven • timeline-demo  
  
Today's pulse  
words: 51  
streak: 0d • active: 24m  
  
Project  
15 paragraphs total  
by status: 15 None  
  
any key dismisses • auto-close in 2s
```

Startup pulse splash — today's words, current streak, active time, by-status counts. Auto-closes after 7s or any key.

The cheat sheet

Ctrl+B H opens the quick-reference overlay — every chord relevant to the current pane:



Ctrl+B H — pane-aware quick reference. Scoped to current focus + the layer-aware chord tables.

The printed cheat sheet (`Documentation/INKHANE_CHEAT_SHEET.typ`) compiles to a two-column A4 PDF with every chord, hook, and CLI subcommand. Print it; hang it next to the keyboard.

Credits

`Ctrl+B V` opens the credits pane — version, dependencies, the embedded logo, and a one-liner about the typst engine in use.

Tutorial-by-tutorial colour examples

The full `Documentation/Tutorials/11-theming.md` walks through every theme key with before/after screenshots. Worth a read once you start customising in earnest.

RECAP

- Theme stanza in HJSON — every colour configurable.

- Per-syntax knobs for editor highlighting; per-lexicon knobs for the overlay.
- `Ctrl+B H` — pane-aware cheat sheet on demand.
- `Documentation/INKHAVEN_CHEAT_SHEET.typ` compiles to a printable two-column A4 reference.
- `Ctrl+B V` — credits + logo.

28

Reassigning keys

Every chord in inkhaven is a binding to a named action like `editor.save` or `view.story_graph`. Bindings live in HJSON; overlays at runtime work too. Re-skin the chord set without recompiling.

The keys stanza

```
keys: {
  bindings: [
    # Move the Bund prefix from Ctrl+Z to Ctrl+B b.
    { chord: "Ctrl+B b", action: "bund.run_buffer" }

    # Bind F11 back to explain-diagnostic (Linux/Windows
    # only – macOS grabs F11).
    { chord: "F11", action: "editor.explain_diagnostic" }

    # Disable a default chord (action: "none").
    { chord: "Ctrl+V w", action: "none" }
  ]
}
```

Each entry is `{ chord, action }`. Multiple bindings to the same action work — both chords fire it.

Chord syntax

Chord	What it does
Letter	Just <code>`a`</code> or <code>`Z`</code> .
Modifier + key	<code>`Ctrl+B`</code> , <code>`Alt+F2`</code> , <code>`Shift+Tab`</code> , <code>`Super+L`</code> .
Modifier + capital	<code>`Shift+W`</code> (the W chord with shift).
Two-step	<code>`Ctrl+B`</code> then <code>`P`</code> — write as <code>`Ctrl+B p`</code> .
F-keys	<code>`F1`</code> through <code>`F12`</code> . <code>`Ctrl+F12`</code> for the modifier variant.

Chord	What it does
Special	`Enter`, `Esc`, `Tab`, `Space`, `BackSpace`, `Up`, `Down`, ...

Listing actions

```
inkhaven keys list
```

Prints every action + its current chord. Useful when you forget the action name.

Layered chord tables

Inkhaven groups chords into layers:

Chord	What it does
TopLevel	Direct chords (Ctrl+S, F7, etc.).
MetaSub	After the Meta prefix (Ctrl+B → action).
BundSub	After the Bund prefix (Ctrl+Z → action).
ViewSub	After the View prefix (Ctrl+V → action).

The `Ctrl+B H` cheat sheet (Chapter 27) shows all four layers grouped — useful when you want to see what chord slots are still free.

Dynamic rebinding via Bund

`ink.key.bind` lets a Bund script rebind at runtime:

```
"Ctrl+B j" "view.fuzzy_paragraph_picker" ink.key.bind
```

Useful in `scripting.bootstrap` for per-project chord adjustments without touching the project-shared HJSON. Or in a hook that reacts to a state change.

Policy: `keymap` (default-denied — opt in via `scripting.enabled_categories`).

Re-binding the prefix keys

The three prefix chords are reconfigurable:

```
keys: {
  meta_prefix: "Ctrl+B"      # default
  bund_prefix:  "Ctrl+Z"      # default
  view_prefix:  "Ctrl+V"      # default
}
```

If `Ctrl+Z` collides with your terminal's suspend chord, move the Bund prefix to `Ctrl+\` or `Alt+B`.

CAVEAT

Changing the prefix doesn't change the sub-chord names in `MetaSub` / `BundSub` / `ViewSub`. Inkhaven's internal action table is keyed by the layer name plus the sub-chord; the prefix is just how the user gets to the layer.

F11 → Ctrl+F12 — the macOS workaround

In 1.2.6, the AI-explain-diagnostic chord moved from `F11` to `Ctrl+F12` because macOS grabs F11 globally (Mission Control / Show Desktop) — the chord never reached the TUI.

Linux + Windows users who prefer F11 can bind it back:

```
keys: {
  bindings: [
```

```
    { chord: "F11", action: "editor.explain_diagnostic" }  
  ]  
}
```

The original Ctrl+F12 binding stays — both fire the action.

RECAP

- Every chord binds an action via `keys.bindings` in HJSON.
- Chord syntax: Ctrl+B p, Shift+W, Ctrl+F12.
- `inkhaven keys list` enumerates actions + current chords.
- Four chord layers: TopLevel, MetaSub, BundSub, ViewSub.
- `ink.key.bind` from Bund rebinds at runtime.
- Prefix chords (Meta / Bund / View) are reconfigurable.

29

Bund scripting

Bund is the embedded scripting language. Stack- based, terse, lambda-friendly. Inkhaven embeds it because the alternative — writing a YAML config language that tries to express everything from a Python — always collapses under its own weight.

You don't have to learn Bund to use inkhaven. You learn it when you want to react to a hook, automate a chord, or write a workflow that's bigger than the chord chart can express.

Where Bund lives

Four touchpoints:

Chord	What it does
<code>scripting.bootstrap</code> (HJSON)	Single string evaluated once at VM init. Natural home for hook lambdas.
Scripts system book	Tree nodes with <code>.bund</code> extension. Inkhaven <code>bund.eval</code> 's every Script node at startup, after the bootstrap.
Ctrl+Z (Bund prefix)	Inside the TUI: Ctrl+Z then <code>`N`</code> creates a Script node; <code>`Ctrl+Z R`</code> runs the current buffer; <code>`Ctrl+Z ?`</code> opens the script picker.
<code>`inkhaven bund "... "`</code>	CLI: one-shot eval against the project's VM.

Stack model

Values flow through the VM's stack. A literal pushes; a word consumes from the top and pushes back its result. So:


```
40 2 +          # pushes 40, pushes 2, + pops two and
pushes 42
"Hello, " "world" cat println  # cat concats, println pops
+ prints
```

Lambdas are wrapped in `{ ... }` and assigned with `define` or registered as hooks with `register`:

```
{ dup * } "square" define
3 square println          # → 9

"hook.on_save" { drop "saved" println } register
```

The full language tour is in `Documentation/Bund/BUND_TUTORIAL.md`.

The ``ink.*`` `stdlib`

Inkhaven exposes the project state through a `stdlib` of `ink.*` words. The full catalog is in Appendix C. The families:

Chord	What it does
<code>ink.node.*</code>	Read the hierarchy: list, get, children.
<code>ink.paragraph.*</code>	Read + write paragraph bodies, status, target, save.
<code>ink.tag.*</code>	List / search / add / remove tags (Chapter 14).
<code>ink.event.*</code>	Add / list / link events (Chapter 17).
<code>ink.story.render</code>	Render the book story view to PNG (Chapter 16).
<code>ink.editor.*</code>	Editor surface: insert, set_cursor, replace, scroll.
<code>ink.key.*</code>	Rebind chords at runtime (Chapter 28).
<code>ink.search.text</code>	Project-wide semantic search (Chapter 10).
<code>ink.fs.*</code>	File-system access — gated by <code>fs_read</code> / <code>fs_write</code> .
<code>ink.ai.*</code>	Trigger AI inference; set system prompts.
<code>ink.input</code>	Pop a modal prompt for the user.
<code>ink.pane.*</code>	Open / write to / close a floating output pane.

Every word has a stack signature documented in `BUND_TUTORIAL.md` and Appendix C.

Hooks

Inkhaven fires named hooks at well-defined points. Register a lambda with the hook's name to react:

Chord	What it does
<code>hook.on_create (uuid kind --)</code>	Node created in the tree.
<code>hook.on_save (uuid --)</code>	Paragraph saved.
<code>hook.on_rename (uuid new_title --)</code>	Node renamed.
<code>hook.on_snapshot (parent_uuid snap_uuid --)</code>	Snapshot taken.
<code>hook.on_delete (uuid --)</code>	Node deleted (once per id in a subtree).
<code>hook.on_status_promoted (uuid new_status --)</code>	Status ladder bumped up.
<code>hook.on_goal_hit (word_count_today --)</code>	Daily goal reached.
<code>hook.on_diagnostic (uuid count first-message --)</code>	Typst diagnostic

Chord	What it does
	state changed (1.2.6+).
<code>hook.on_event_added (uuid --)</code>	Time-line event created (1.2.6+).
<code>hook.on_event_orphaned (uuid --)</code>	Event lost its last link (1.2.6+).

Sandbox policy

Bund runs inside a policy sandbox. Categories default-deny the destructive parts:

Chord	What it does
<code>store_read</code>	Default-allowed. <code>ink.node.*</code> , <code>ink.search.*</code> , etc.
<code>store_write</code>	Default-denied. <code>ink.paragraph.save</code> , <code>ink.tree.delete</code> , etc.
<code>fs_read</code>	Default-allowed.
<code>fs_write</code>	Default-denied. <code>ink.fs.write</code> , <code>ink.story.render</code> to disk.
<code>net</code>	Default-denied. <code>ink.net.*</code> .
<code>shell</code>	Default-denied. <code>ink.shell.*</code> .
<code>code_eval</code>	Default-denied. Bund's own <code>`eval`</code> word.
<code>keymap</code>	Default-denied. <code>ink.key.bind</code> at runtime.
<code>ai_write</code>	Default-denied. <code>ink.ai.set_system_prompt</code> .
<code>editor_write</code>	Default-allowed. <code>ink.editor.*</code> (the editor is yours).
<code>theme_write</code>	Default-denied. <code>ink.theme.*</code> .

Opt in via HJSON:

```
scripting: {
  enabled_categories: ["store_write", "fs_write"]
  no_default_deny:    false
}
```

`no_default_deny: true` clears the entire deny list (use sparingly — you're trusting every Bund script you run).

A first hook

A typical bootstrap that auto-snapshots when a paragraph promotes to Final:

```
scripting: {
  enabled_categories: ["store_write"]
  bootstrap: '''
    "hook.on_status_promoted" {
      // ( uuid new_status -- )
      "Final" =
      {
        // Find the open paragraph; snapshot.
        // (Pseudo — see BUND_TUTORIAL.md for snapshot Bund
words.)
        drop "promoted to Final" println
      } { drop drop } ifelse
    } register
  '''
}
```

Where to learn more

- `Documentation/Bund/BUND_TUTORIAL.md` — language tour + every stdlib word.
- Appendix C — quick reference card for the `ink.*` stdlib
 1. hooks.

- Real example scripts ship in `Documentation/Bund/`.

RECAP

- Four touchpoints: `scripting.bootstrap`, Scripts book, `Ctrl+Z` chord, `inkhaven bund`.
- Stack-based, terse, lambdas with `{ ... }`.
- `ink.*` `stdlib` exposes `node / paragraph / tag / event / story / editor / search / AI / fs / input / pane`.
- Hooks: `on_save`, `on_rename`, `on_status_promoted`, `on_diagnostic`, `on_event_added`, etc.
- Policy categories default-deny destructive surfaces; opt in via `enabled_categories`.

APPENDIX A



Keybinding reference

Every chord that ships with inkhaven, by layer. The canonical machine-readable list lives at `Documentation/KEYBINDING.md`; this appendix is the printed companion.

Global (anywhere)

Chord	What it does
Ctrl+Q	Quit (autosaves dirty paragraph).
Tab / Shift+Tab	Cycle Tree → Editor → AI.
Ctrl+1/2/3/4/5	Focus Editor / Tree / AI / Search / AI prompt.
Ctrl+T	Focus Tree.
Ctrl+S	Save current paragraph.
Ctrl+/	Focus search bar.
Ctrl+I	Focus AI prompt.
Ctrl+B	Meta prefix (next key = action).
Ctrl+Z	Bund prefix.
Ctrl+V	View prefix.
F1	Help RAG query.
F7	Grammar check (editor scope).
F8	Typst diagnostics list (1.2.6+).
F9 / Shift+F9	Cycle AI scope forward / backward.
F10	Toggle inference mode (Local ↔ Full).
F12	AI critique — mode-aware (1.2.6+).
Ctrl+F12	AI explain diagnostic (1.2.6+ — was F11).
Esc	Close overlay / cancel.

Tree pane

Chord	What it does
↑ / ↓ / Home / End	Navigate.
PgUp / PgDn	Jump by 10.
← / →	Collapse / expand branch.
Enter	Open paragraph.
F2	Rename current node.
F3	File picker — load file / import tree.
B / C / A / +	Add book / chapter / subchapter / paragraph.
V / S / P	Insert chapter / subchapter / paragraph after current.
D	Delete branch (confirm).
-	Delete paragraph at cursor.
U / J	Reorder up / down among siblings.
Z / X	Collapse subchapter / collapse every branch.
Space	Multi-select toggle.
T	Cycle type (works on marks).
O	Cycle status (works on marks).
g	Tag picker (works on marks).

Editor pane

Chord	What it does
Ctrl+S	Save.
F5	Snapshot with annotation prompt (1.2.6+).
F6	Snapshot picker.
F4	Toggle split-edit.
Ctrl+F4	Accept split-edit snapshot as the new baseline.
F7 / F12 / Ctrl+F12	Grammar / critique / explain (see Global).
F8	Diagnostics list (1.2.6+).
Ctrl+F	Find in buffer.
Ctrl+H	Find and replace in buffer.

AI pane

Chord	What it does
r / R	Replace buffer (routes through diff modal).
g / G	Replace with grammar-corrected text only.
i / I	Insert at cursor.
t / T	Prepend (top).
b / B	Append (bottom).
c / C	Copy to clipboard.
Ctrl+F	Search chat history.
Ctrl+C	Enter selection mode.

Meta prefix (Ctrl+B)

Chord	What it does
Ctrl+B A	Schedule assemble.
Ctrl+B B	Schedule build (PDF).
Ctrl+B O	Schedule extra-format builds.
Ctrl+B C	Clear chat history.
Ctrl+B G	Notes RAG.
Ctrl+B H	Cheat-sheet overlay.
Ctrl+B I	Book info modal.
Ctrl+B K	Toggle AI full-screen.
Ctrl+B L	LLM provider picker.
Ctrl+B M	Show inference mode.
Ctrl+B N / U / P / C / A	Open Notes / Research / Places / Characters / Artefacts listing or RAG.
Ctrl+B R / Shift+R	Cycle status forward / backward.
Ctrl+B 1..7	Status filter modal.
Ctrl+B V	Credits.

Chord	What it does
Ctrl+B] / }	Tag picker / tag search (1.2.5+).

Bund prefix (Ctrl+Z)

Chord	What it does
Ctrl+Z R	Run current buffer as Bund.
Ctrl+Z N	New Script node.
Ctrl+Z E	Open eval modal.
Ctrl+Z ?	Script picker.

View prefix (Ctrl+V)

Chord	What it does
Ctrl+V 1 / 2	Markdown extract: paragraph / subchapter.
Ctrl+V S	Toggle similar-paragraph mode.
Ctrl+V G	Open progress modal.
Ctrl+V t / Shift+T	Per-¶ word-count target / open timeline view (1.2.6+).
Ctrl+V A / I	Add outgoing / incoming link.
Ctrl+V L / K	List outgoing / incoming links.
Ctrl+V B / M	Toggle / list bookmarks.
Ctrl+V P	Fuzzy paragraph picker.
Ctrl+V R	Render paragraph preview.
Ctrl+V N / Shift+N	Next / previous diagnostic.
Ctrl+V w / Shift+W	Paragraph mini story view / book story view (1.2.6+).
Ctrl+V e	Event picker (1.2.6+).
Ctrl+V Shift+E	New event from any pane (1.2.6+).
Ctrl+V Shift+I	Edit open event's start end track (1.2.6+).

Inside the timeline view (Ctrl+V Shift+T)

Chord	What it does
← / → / PgUp / PgDn	Scroll.
+ / = / - / _	Zoom in / out.
0	Reset zoom.
Home / End	Jump to first / last event.
u / U / d / D / b / B / p / P	Up / down / book / project scope.
Tab	Cycle highlighted track.
Enter	Open the event closest to cursor.
n / N	New event at cursor tick.
y / Y / Ctrl+Y	AI critique — track / scope / book-wide.

Inside Ctrl+V R (render preview)

Chord	What it does
← / → / Home / End	Page navigation.
+ / = / - / _	Zoom (1.2.6+).
0	Reset zoom.
S	Save current page.
A	Save every page.

B

Configuration reference

Every HJSON knob, by stanza. The canonical machine-readable reference lives at `Documentation/CONFIGURATION.md`; this appendix is the condensed printed companion.

Top-level

```
{
  language: "english"
  prompts_file: "prompts.hjson"
  artefacts_directory: ""           # empty → sibling
  sync_interval_seconds: 5
}
```

editor

```
editor: {
  typewriter_sounds:      false
  autosave_idle_seconds:  30
  word_wrap:              true
  startup_splash:         true
  paragraph_target_default: 250
  stemming: { languages: ["english"] }
}
```

llm — AI providers

```
llm: {
  default_provider: "ollama"
```

```
ollama: { model: "qwen2.5:7b", host: "http://localhost:
11434" }
gemini: { model: "gemini-2.0-flash-exp" }
claude: { model: "claude-sonnet-4-6" }
openai: { model: "gpt-4o-mini" }
deepseek:{ model: "deepseek-chat" }
grok:    { model: "grok-2-latest" }
}
```

API keys come from environment variables: `GEMINI_API_KEY`, `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `DEEPSEEK_API_KEY`, `XAI_API_KEY`. Ollama needs none.

ai (1.2.6+)

```
ai: {
  per_paragraph_memory:      false
  per_paragraph_memory_max_turns: 10
  diff_review_on_apply:      true
  reseed_prompt_examples:    true
}
```

typst_compile

```
typst_compile: {
  engine: "external"           # or "inprocess"
  bundle_fonts: true
  use_system_fonts: true
  packages_enabled: true
  diagnostics_idle_seconds: 2.0
  diagnostics_max: 50
}
```

typst_page / typst_fonts / typst_layout

```
typst_page: {  
  paper: "us-letter"           # or "a4", "iso-b5", ...  
  margin: { top: "2.5cm", bottom: "2.5cm",  
            left: "2cm", right: "2cm" }  
}  
  
typst_fonts: {  
  body: "Linux Libertine"  
  size: "11pt"  
}  
  
typst_layout: {  
  par: { justify: true, indent: "1em" }  
}
```

images

```
images: {  
  preview_enabled: true  
}
```

False when the host terminal can't render images — kills the half-block fallback that's usually worse than no preview.

output

```
output: {  
  extra_formats: ["markdown", "epub"]  
}
```



```
epub_author: "Vladimir Ulogov"
}
```

goals

```
goals: {
  daily_words: 800
  morning_baseline: true
  streak_grace: 1
  per_book_deadline: {
    "my-first-book": "2026-08-31"
  }
}
```

timeline (1.2.6+)

```
timeline: {
  enabled: true
  default_track: "main"
  calendar: { preset: "gregorian" }    # or "sols" /
"custom"
  display: {
    show_orphans: true
    swim_lane_max_rows: 12
    default_zoom: 1.0
  }
}
```

Custom calendar block — see Chapter 17 + 5 for the full shape.

backup

```
backup: {
  out_dir: ""           # empty → sibling
  max_age: "24h"        # "0s" disables auto-backup
}
```

sound

```
sound: {
  typewriter_enabled: false
  return_chime: true
}
```

scripting

```
scripting: {
  enabled_categories: []
  no_default_deny: false
  bootstrap: ""
}
```

Categories: `store_read`, `store_write`, `fs_read`, `fs_write`, `net`, `shell`, `code_eval`, `keymap`, `ai_write`, `editor_write`, `theme_write`. Default deny: `fs_write`, `net`, `shell`, `code_eval`, `keymap`.

keys

```
keys: {
  meta_prefix: "Ctrl+B"
  bund_prefix: "Ctrl+Z"
  view_prefix: "Ctrl+V"
  bindings: [
```

```
# { chord, action } pairs
]
}
```

theme

The full list of theme keys is long; see [Documentation/CONFIGURATION.md](#). Most-used:

```
theme: {
  bg: "#1e1e2e"
  fg: "#cdd6f4"
  current_line_bg: "#313244"
  selection_bg: "#414559"
  status_bar_bg: "#181825"
  modal_border: "#cba6f7"
  modal_bg: "#181825"
  modal_fg: "#cdd6f4"
  tree_book_fg: "#cba6f7"
  tree_chapter_fg: "#f9e2af"
  tree_subchapter_fg: "#fab387"
  tree_paragraph_fg: "#cdd6f4"
  tree_script_fg: "#cba6f7"
  tree_image_fg: "#94e2d5"
  tree_open_marker: "#a6e3a1"
  editor_character_fg: "#94e2d5"
  editor_place_fg: "#f9e2af"
  editor_artefact_fg: "#cba6f7"
  syntax_heading: "#f38ba8"
  syntax_emphasis: "#cdd6f4"
  syntax_strong: "#fab387"
  syntax_function: "#89dceb"
  syntax_operator: "#94e2d5"
  syntax_list_marker: "#cba6f7"
  syntax_raw: "#fab387"
  syntax_tag: "#89b4fa"
  syntax_quote: "#9399b2"
```

```
line_number_fg:    "#5d5d5d"  
grammar_change_fg: "#a6e3a1"  
status_napkin:     "#9399b2"  
status_first:      "#fab387"  
status_second:     "#f9e2af"  
status_third:      "#a6e3a1"  
status_final:      "#89dceb"  
status_ready:      "#cba6f7"  
}
```

APPENDIX C



Bund stdlib + hooks

Quick reference for the `ink.*` stdlib and the hook lambda names. Stack signatures only — the full language tutorial lives at `Documentation/Bund/BUND_TUTORIAL.md`.

Tree (`ink.node.*`, `ink.tree.*`)

Chord	What it does
<code>ink.node.list</code>	(-- list)
<code>ink.node.get</code>	(uuid -- hash NODATA)
<code>ink.node.children</code>	(parent_uuid_or_empty -- list)
<code>ink.path.to_uuid</code>	(path -- uuid NODATA)
<code>ink.tree.add</code>	(parent_path kind title -- uuid)
<code>ink.tree.delete</code>	(uuid --)
<code>ink.tree.rename</code>	(uuid new_title --)
<code>ink.tree.move_up / move_down</code>	(uuid --)
<code>ink.tree.morph</code>	(uuid new_kind --)

Paragraph (`ink.paragraph.*`)

Chord	What it does
<code>ink.paragraph.text</code>	(uuid -- string NODATA)
<code>ink.paragraph.save</code>	(path body --)
<code>ink.paragraph.set_status</code>	(path status --)
<code>ink.paragraph.set_target</code>	(path int --)
<code>ink.paragraph.target</code>	(path -- int NODATA)

Tags (**ink.tag.***, 1.2.6+)

Chord	What it does
<code>ink.tag.list</code>	(-- list)
<code>ink.tag.list_for</code>	(path -- list NODATA)
<code>ink.tag.search</code>	(tag -- list-of-paths)
<code>ink.tag.add</code>	(path tag --)
<code>ink.tag.remove</code>	(path tag --)

Events (**ink.event.***, 1.2.6+)

Chord	What it does
<code>ink.event.list</code>	(-- list)
<code>ink.event.list_orphans</code>	(-- list)
<code>ink.event.add</code>	(book title spec -- uuid)
<code>ink.event.set_end</code>	(uuid spec --)
<code>ink.event.set_precision</code>	(uuid prec --)
<code>ink.event.set_track</code>	(uuid track --)
<code>ink.event.link_paragraph</code>	(uuid path --)

Story (**ink.story.***, 1.2.6+)

Chord	What it does
<code>ink.story.render</code>	(book-name path --)

Stack-only output; writes the PNG to `path`. Policy: `fs_write`.

Editor (ink.editor.*)

Chord	What it does
<code>ink.editor.cursor</code>	(-- list[row col] NO- DATA)
<code>ink.editor.goto</code>	(row col --) // 0- based
<code>ink.editor.set_cursor</code>	(row col --) // 1- based (1.2.6+)
<code>ink.editor.insert</code>	(text --)
<code>ink.editor.replace</code>	(text --)
<code>ink.editor.replace_all</code>	(pat- tern re- place- ment --)
<code>ink.editor.delete_line / delete_to_bol / delete_to_eol</code>	(--)
<code>ink.editor.scroll</code>	(delta_rows --)

Search (ink.search.*)

Chord	What it does
ink.search.text	(query limit -- list)

Snapshots (ink.snapshot.*)

Chord	What it does
ink.snapshot.list	(uuid -- list)

Filesystem (ink.fs.*)

Chord	What it does
ink.fs.read	(path -- string)
ink.fs.write	(path body --) // fs_write policy

AI (ink.ai.*)

Chord	What it does
ink.ai.send_blocking	(prompt -- response) // ai_write policy
ink.ai.set_system_prompt	(prompt --) // ai_write policy

Keymap (ink.key.*)

Chord	What it does
ink.key.bind	(chord action --) // keymap policy
ink.key.unbind	(chord --) // keymap policy

Pane + input

Chord	What it does
ink.pane.open	(title -- handle)
ink.pane.write	(handle text --)
ink.pane.close	(handle --)
ink.input	(prompt -- string) // pops a modal

DB (ink.db.*)

Chord	What it does
ink.db.sync	(--)
ink.db.checkpoint	(--)
ink.db.reindex	(--)

Hooks (fire signatures)

Each hook is registered with `register`. The body sees the fire-time arguments on the stack, bottom-first.

Chord	What it does
hook.on_create	(uuid kind --)
hook.on_save	(uuid --)
hook.on_rename	(uuid new_title --)
hook.on_snapshot	(parent_uuid snapshot_uuid --)
hook.on_delete	(uuid --) // per id in subtree
hook.on_status_promoted	(uuid new_status --)
hook.on_goal_hit	(word_count_today --)
hook.on_streak_break	(--)
hook.on_assemble	(book_uuid path --)
hook.on_take	(book_uuid pdf_path --)
hook.on_diagnostic	(uuid count first-message --) // 1.2.6+
hook.on_event_added	(uuid --) // 1.2.6+
hook.on_event_orphaned	(uuid --) // 1.2.6+

Policy categories

Chord	What it does
store_read	Default allowed. Tree + paragraph + event reads.
store_write	Default denied. ink.tree.*, ink.paragraph.save, ink.event.add/set_*, ink.tag.add/remove.
fs_read	Default allowed. ink.fs.read.
fs_write	Default denied. ink.fs.write, ink.story.render.
net	Default denied.
shell	Default denied.
code_eval	Default denied. Bund's `eval`.
keymap	Default denied. ink.key.bind.
ai_write	Default denied. ink.ai.*.
editor_write	Default allowed. ink.editor.*.
theme_write	Default denied.

About the author



Vladimir Ulogov.

Vladimir Ulogov has spent decades building infrastructure for distributed systems — the kind of software that watches other software. Early in his career he worked on monitoring and telemetry platforms; later years took him into federated observability, telemetry buses, and the architecture of systems that have to make sense of millions of data points without losing the thread.

He is also a familiar face on the conference circuit — talks on observability design, on the discipline of knowing what a system is doing without drowning in metrics, on building software that can be read as well as run.

The same questions keep coming back: what deserves to be measured, what deserves to be ignored, and how does the tool stay out of its own way.

What makes him slightly unusual in his corner of the industry is a tendency to write his own tools — not in the sense of small utilities, but in the sense of programming languages. The Bund language (its compiler, its VM, its document store, its parser) lives in a long series of Rust crates on crates.io. `rust_dynamic`, `rust_multistack`, `rust_multistackvm`, `bundcore`, `zbus_universal_data_gateway` — each is a building block that exists because the off-the-shelf options didn't fit the shape of the work.

Why Inkhaven exists

Inkhaven is Vladimir's personal reflection on how a literary tool can help the people who write books. It grew out of two things, neither of them grand.

The first was his own experiments in literature — fiction drafts, essays, the long slow project of figuring out how his prose wants to behave on the page. A working editor is a piece of furniture you sit in for hours, and the editor he wanted didn't exist in the shape he wanted it.

The second was dissatisfaction. Existing tools — the commercial ones in particular — kept getting the small things wrong. The tree pane didn't model the way manuscripts actually nest. The search was either too literal or too loose. Snapshots were an afterthought. Tags were a checkbox. AI was either everywhere and intrusive or entirely absent. None of these are world-shaking problems on their own, but added together they make the hours stack up against the writing instead of with it.

Inkhaven was built to make those hours stack up in the other direction. The tree is structural. The search is semantic and exact in parallel. Snapshots get annotations. Tags are a project-wide metadata layer. AI sits behind a chord and a scope flag; it never gets in the way unless you ask. Every feature in this book is one author's answer to one small, persistent thing he wanted out of the work.

A work of love

Inkhaven is open source. The licence is permissive — you can read it, fork it, study it, modify it, and pass it on. Strictly speaking, the licence also lets you sell it. The author would, gently but firmly, disagree with you doing that. Inkhaven was not designed as a *for sale* project. It is a work of love made for the authors who can least afford to pay for software, and turning it into a commercial product would betray the reason it exists. So please — don't.

It carries no analytics, no telemetry, no upsell. The binary will never phone home; the project will never have an “Enterprise” tier you have to escape from.

This was a deliberate choice. There are excellent commercial editors for writing books. They cost money — which is fine for many writers, and a barrier for many more. Inkhaven exists for the second group: for the graduate student writing a dissertation on a battered laptop, for the novelist who shouldn't have to pick between rent and software, for the engineer drafting in the same terminal where they already write code, for anyone who would benefit from a tool that respects their work without asking for a credit-card number.

It is not built to compete with those other editors. It is not a Scrivener-killer, a Vellum-killer, a Word-killer. It's a quieter project that says: here is another way; if it fits your hands, use it.

The author calls inkhaven a *work of love* — which is a phrase that sometimes embarrasses people in software circles, but which means precisely what it says. The hours spent on it weren't pulled from a balance sheet. The features weren't road-mapped. The choices that shaped it were the choices of someone who hoped, in writing each feature, that the next writer who opened it would feel slightly less alone in front of an empty paragraph.

A note on cooperation

Vladimir believes firmly in the human capacity for mutual help — that we make better work, and live better lives, when we share what we know and what we build. Software written in that spirit is a small contribution to that

larger pattern. Open source is one of the most concrete expressions of cooperation our era has produced: code read, improved, and passed forward without payment, without permission, by people who will never meet.

If inkhaven helps you finish your book — that is enough. If it gives you a chord pattern you adapt into your own tool — that’s a gift back to the larger project of making software writers can love. As a society, we achieve the greatest things when we help each other rather than compete with each other.

The book in your hands is part of that hope. Read it, disagree with it, fork it, send a pull request, file an issue — whatever shape your contribution takes, the project is large enough to hold it.

Where to find more

Chord	What it does
GitHub	@vulogov — the source for inkhaven, Bund, and the dozen-plus Rust crates that carry the infrastructure. Issues and PRs welcome.
LinkedIn	/in/vladimirulogov — posts on observability, the occasional long-form essay.
YouTube	@vulogov — talks and walkthroughs from the conference trail.
X / Twitter	@vladimir_ulogov

If you build something on top of Inkhaven, or if a chord trips you up and you can't find the answer in this book — open an issue on GitHub. The author reads them.